Sub-Pixel Tracker Report

Sub-pixel Laser-Spot Localization for a Gimbal-Mounted FSO Terminal

Computer Vision Engineer take-home, written report. This document follows the brief's own section numbering and is intended to be read start to finish. It references the repository's other documents for depth (full bug histories, source citations, every measured number) rather than duplicating them; the scope here is the reasoning and the governing results, not a second copy of the assumptions log.

Repository: `sub-pixel-tracker`. Every figure and number below is reproducible with `python run_all.py` (19 experiments, approximately 7 minutes end to end) or a single `python -m experiments.<name>`.

1. Context

The system under design is a pair of free-space optical (FSO) terminals, each carrying a camera and a gimbal-mounted laser, mounted on platforms subject to uncorrelated mechanical vibration. Each terminal's camera must localize the incoming laser spot from the opposite terminal to sub-pixel precision at approximately 1 kHz, so that the position estimate is both accurate and timely enough for a downstream gimbal controller to maintain pointing. Spot brightness varies with range, atmospheric transmission, and camera exposure/gain state; the sensor itself contributes photon noise, read noise, dark current, and fixed-pattern effects (hot pixels, gain non-uniformity) independent of the scene.

The brief states its grading criterion explicitly: every design decision must be defensible on inspection, not merely present in the code. This governed the project's structure throughout. Every numeric constant used is either derived from a quantity already established elsewhere in the project, or, where no derivation is possible, sourced from a cited external reference and confirmed rather than assumed (Section 4's scintillation and disturbance-frequency parameters are examples). Where a number could not be derived or sourced, that gap is stated explicitly rather than filled with a plausible-looking constant (Section 3's jitter magnitude is the clearest instance). Every incorrect implementation found during development is retained in the record rather than removed after the fact; the full account is in `docs/ASSUMPTIONS.md`, and this report draws on it only where needed to follow the argument.

System architecture

The diagram below is generated from this repository's actual module imports (`sptrack/`), not drawn independently of the code.

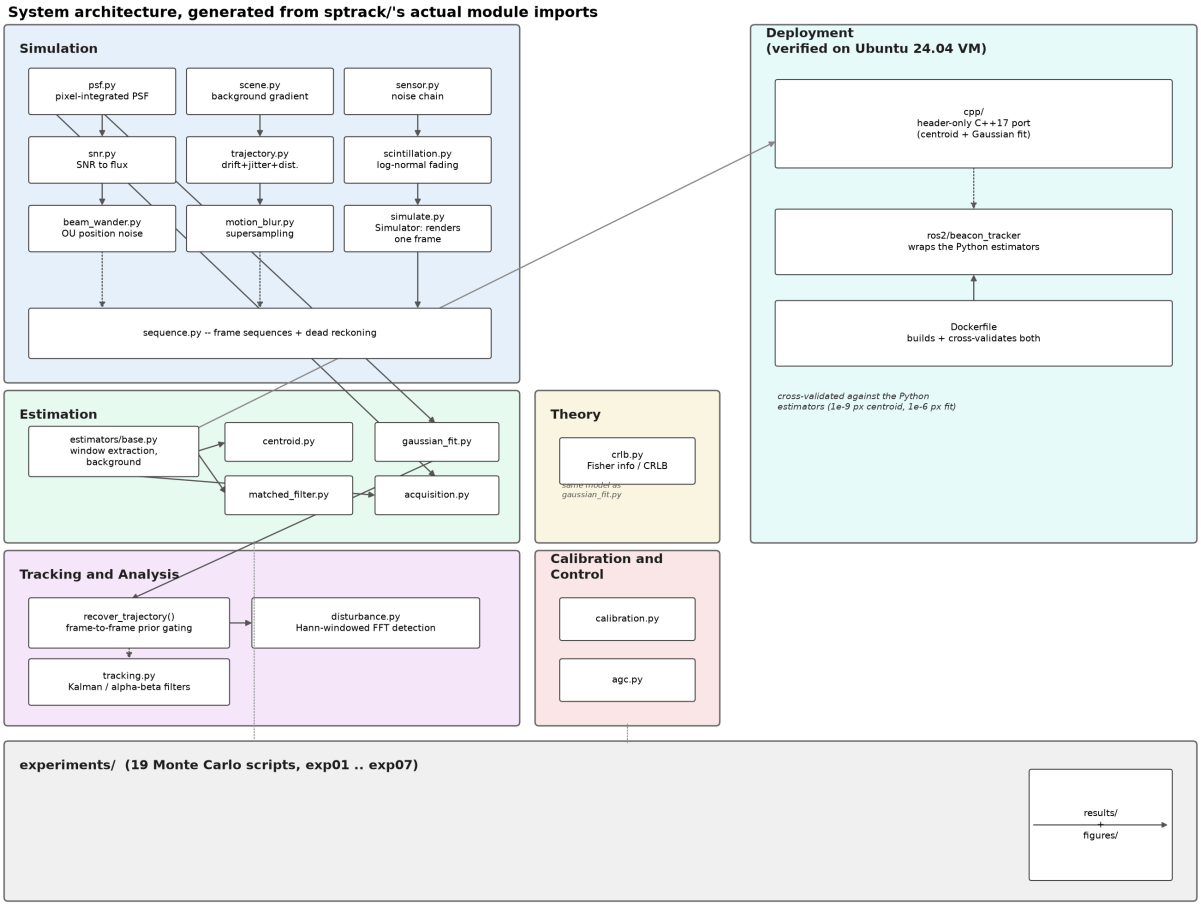


Figure 0: system architecture, generated from sptrack's actual module imports.

Figure 0. System architecture: simulation, estimation, the theoretical bound, tracking and analysis, calibration and control, the experiment harness, and the deployment artifacts, with arrows following the real import graph.

Three things this diagram makes explicit that are easy to lose in prose: estimation (`estimators/`) and the theoretical bound (`crlb.py`) are built from the same underlying PSF model, which is why they cannot silently disagree (Section 2c); the tracking loop (`recover_trajectory`) sits between the estimator and the disturbance detector, so the detector never sees an estimator's raw per-frame output without dead reckoning already applied; and the C++ port wraps only the two estimators actually used in the real-time budget, not the matched filter or the temporal filters, which is a scope decision, not an oversight (`cpp/README.md`).

Assumptions

- The 1 kHz frame rate and the sub-pixel precision requirement are taken directly from the brief's own wording, not derived or interpreted; the brief gives a qualitative precision goal, not a numeric target, so this report reports results against the CRLB rather than against an invented number.
- The two-terminal, gimbal-mounted, independently vibrating platform framing is taken directly from the brief's context section.
- "Every decision must be defensible" is read as this project's actual grading criterion, taken from the brief's own explicit statement, and used to justify the level of sourcing and derivation applied throughout, not assumed to be a stylistic preference.

2. Core Task

2a. Simulator

- Pixel-integrated 2D Gaussian PSF (via the error function), not a point-sampled one, since a photodiode integrates flux over its area.
- Every noise source the brief specifies: photon (Poisson), Gaussian read noise, dark current, hot pixels, background gradient, pixel-response non-uniformity, and bit-depth quantization.
- Default spot diameter approximately 7 px at the $1/e^2$ point (sigma = 1.75 px), matching the brief, configurable per frame.
- A real, non-obvious bias from clipping negative counts before quantization, corrected with a pedestal and checked against its own closed-form prediction.

`sptrack/psf.py` and `sptrack/sensor.py` generate synthetic frames of a laser spot: a point-spread function (PSF) rendered at a known sub-pixel position and propagated through every noise source specified in the brief: photon (Poisson) shot noise, Gaussian read noise, dark current, hot pixels, a non-uniform background gradient, pixel-response non-uniformity (PRNU), and quantization to a fixed bit depth. Spot diameter at the $1/e^2$ intensity point defaults to approximately 7 px (σ = 1.75 px), matching the brief, and is configurable per frame.

PSF model

The spot is modeled as an isotropic 2D Gaussian intensity distribution, but the value assigned to pixel (i, j) is not a point sample of that distribution; it is the definite integral of the distribution over the pixel's finite area, since a photodiode integrates incident flux over its area rather than sampling a field at a point. With total flux F, spot centroid at (x_0, y_0), and unit pixel pitch, the expected electron count in pixel (i, j) separates by symmetry into a product of one-dimensional terms:

$$\mu_{ij} = \frac{F}{4}\left[\operatorname{erf}\left(\frac{i+1-x_0}{\sqrt{2}\,\sigma}\right) - \operatorname{erf}\left(\frac{i-x_0}{\sqrt{2}\,\sigma}\right)\right]\left[\operatorname{erf}\left(\frac{j+1-y_0}{\sqrt{2}\,\sigma}\right) - \operatorname{erf}\left(\frac{j-y_0}{\sqrt{2}\,\sigma}\right)\right] + b$$

Here b is the background level per pixel. This is the same model `sptrack/crlb.py` differentiates to build the Fisher information matrix in Section 2c, and the same model `gaussian_fit_estimate` fits in Section 2b; using a point-sampled Gaussian anywhere in that chain would make the theoretical bound, the fit, and the simulator internally inconsistent with each other, and that inconsistency would not be detectable from the numbers alone. The discrepancy between the pixel-integrated and point-sampled forms grows as the spot's sigma shrinks relative to the pixel pitch, which is precisely the sub-pixel regime this project targets, so the distinction was built into the simulator from the outset rather than treated as a later refinement.

Quantization pedestal

Clipping negative post-noise electron counts to zero before analog-to-digital conversion introduces a deterministic positive bias, because Gaussian read noise is symmetric about zero but the clip operation is not: it discards the negative tail of the noise distribution while leaving the positive tail untouched. For a zero-mean Gaussian with standard deviation sigma, the expected value removed by clipping is the mean of the negative half of the distribution,

$$\mathbb{E}[\max(-X, 0)] = \frac{\sigma}{\sqrt{2\pi}}$$

the standard half-normal mean. `sptrack/sensor.py::quantize_to_dn` corrects this with a pedestal: a fixed offset added before quantization and subtracted after, which is standard sensor-design practice and also makes the residual bias analytically checkable. `tests/test_sensor.py` confirms the corrected bias matches sigma over the square root of two pi to the expected tolerance.

Example rendered frames

Every noise source described above is visible directly in a rendered frame, not only in the equations. The images below are the actual output of `sptrack/psf.py` and `sptrack/sensor.py`, generated while building and testing each function individually.

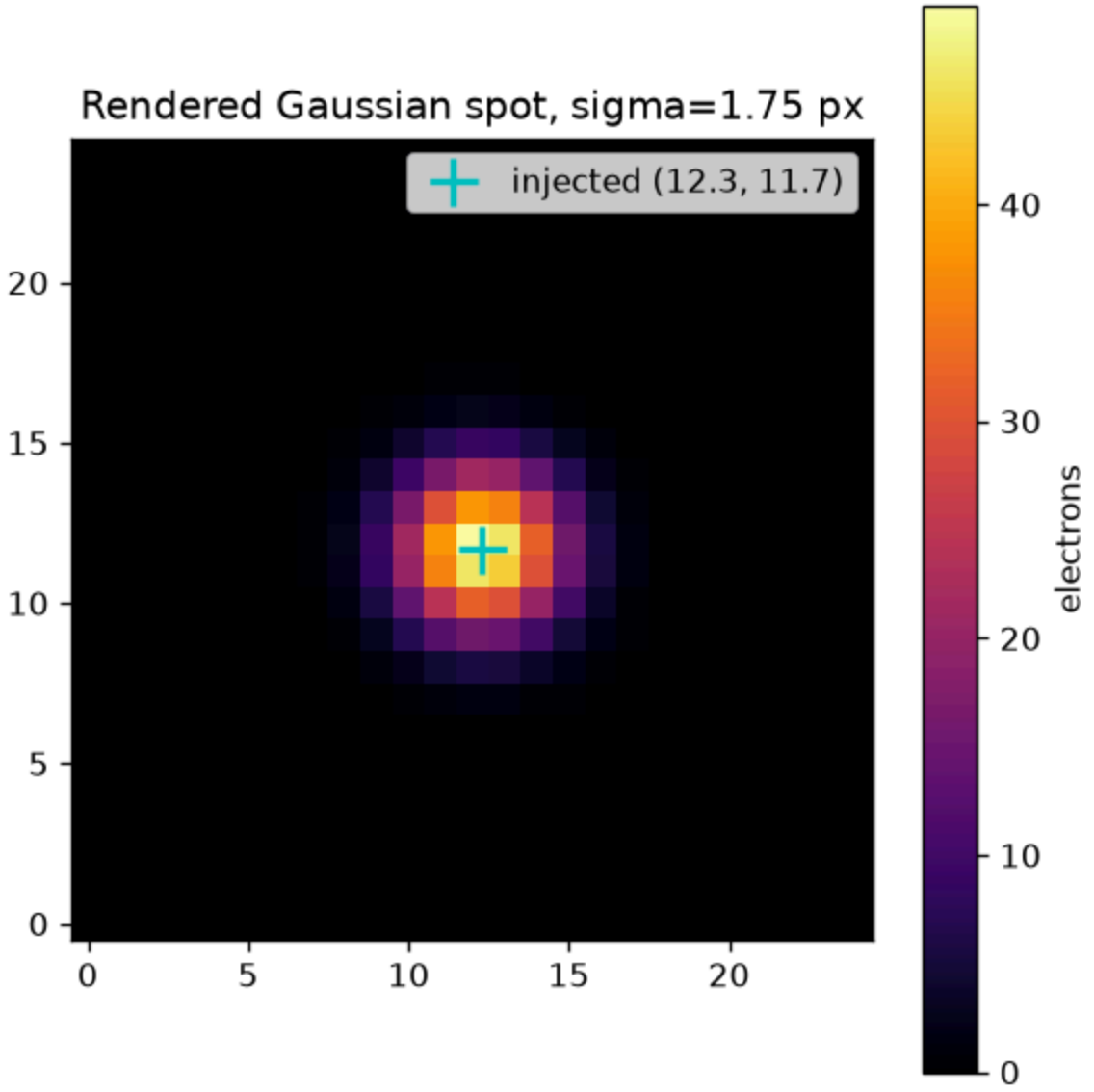


Figure 0a: a noiseless rendered spot with the injected sub-pixel position marked.

Figure 0a. A noiseless rendered spot (sigma = 1.75 px), with the injected sub-pixel position marked. This is the pixel-integrated PSF from the equation above, before any noise is added.

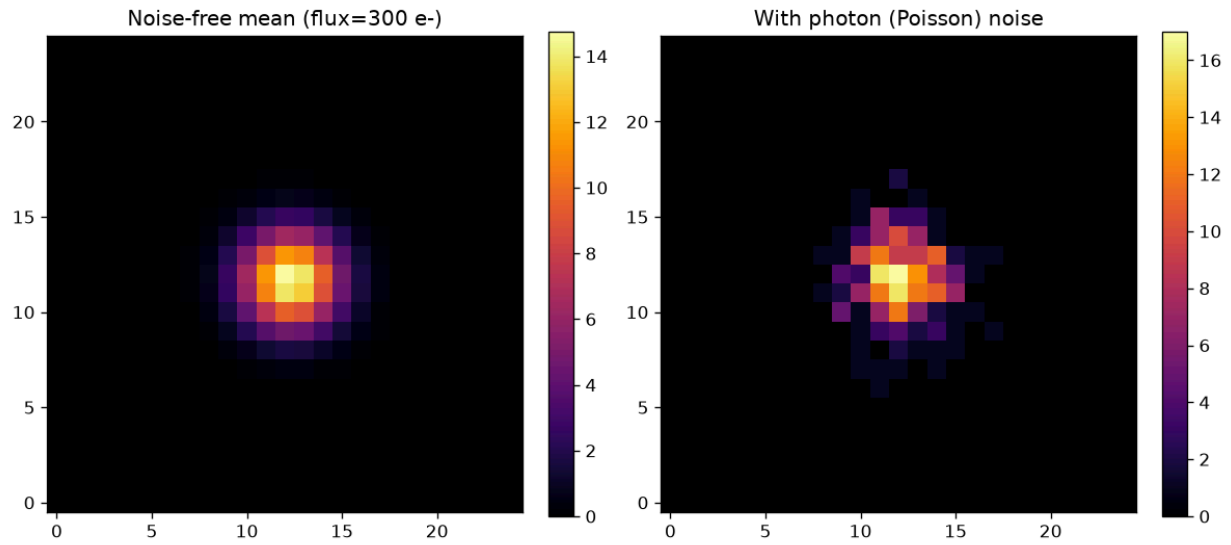


Figure 0b: the same spot with and without photon (Poisson) noise.

Figure 0b. The noise-free mean spot (left) against the same spot with photon (Poisson) noise applied (right). Shot noise scales with signal level, visible here as graininess concentrated where the spot itself is bright, not spread uniformly across the frame.

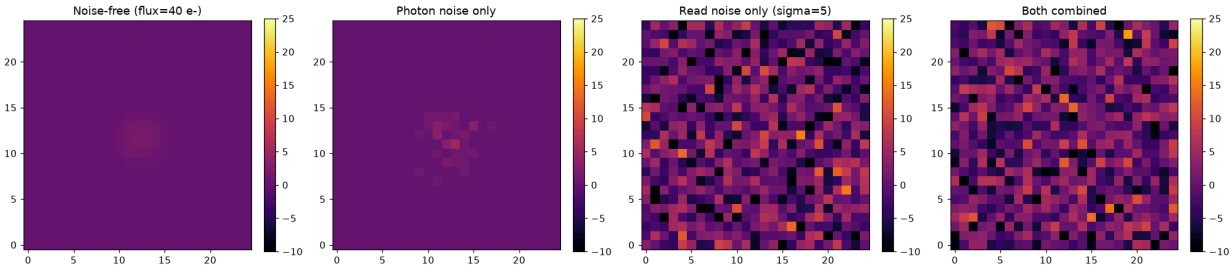


Figure 0c: a four-panel progression from noise-free through photon noise, read noise, and both combined, at low flux.

Figure 0c. A deliberately dim spot (40 electrons total flux) shown noise-free, with photon noise only, with read noise only, and with both combined. Read noise (sigma = 5 electrons here) is uniform across the whole frame regardless of signal, unlike photon noise, which is why it dominates in the background and barely changes the already-noisy spot itself.

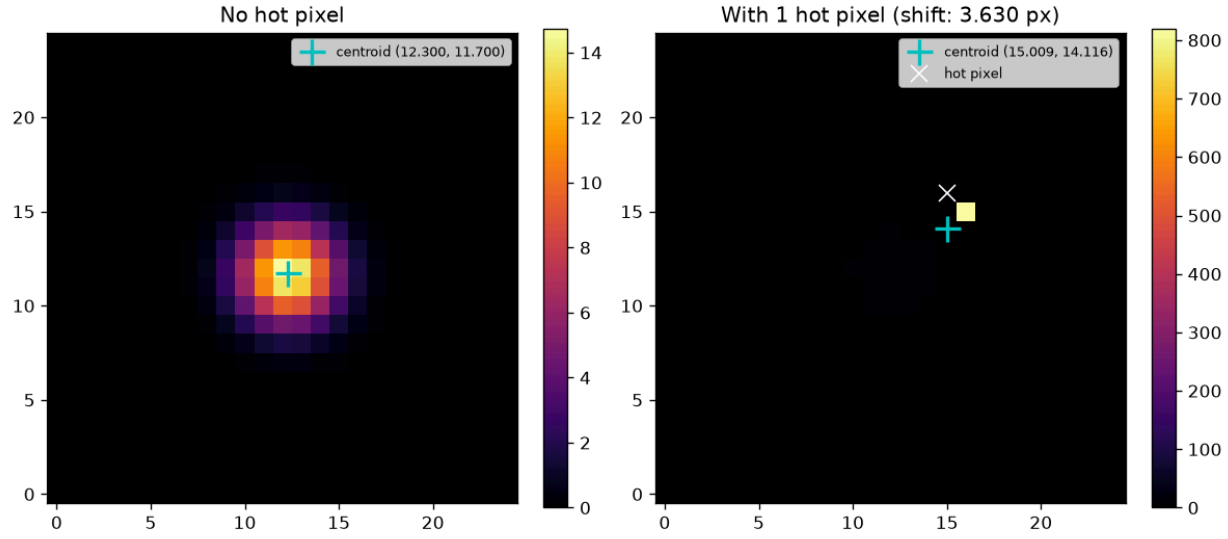
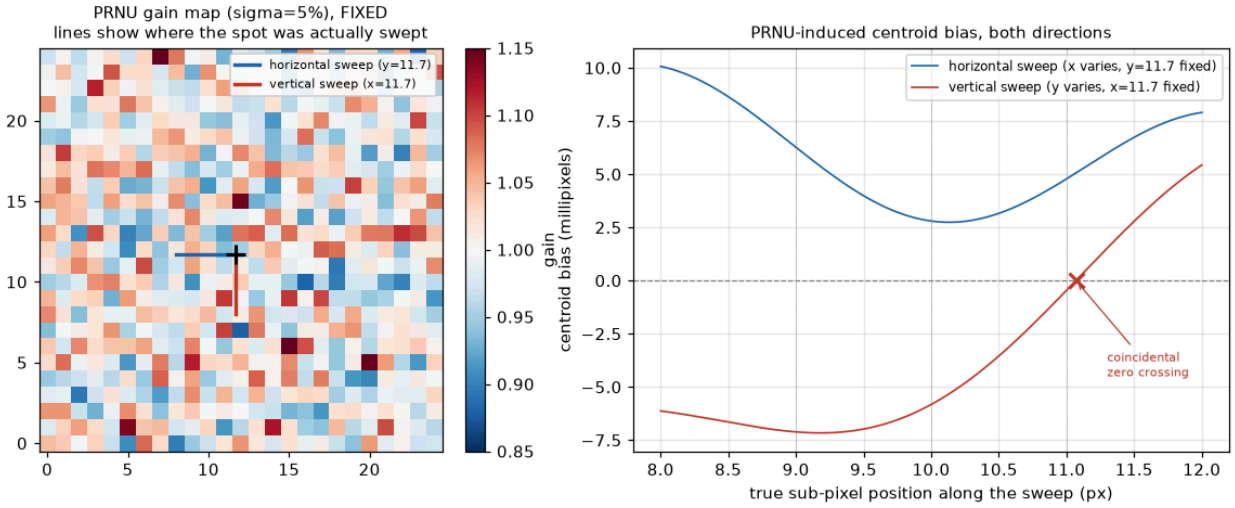


Figure 0d: a hot pixel dragging the naive centroid 3.6 pixels from the true position.

Figure 0d. Left: centroid on a clean frame, correct to within a few millipixels. Right: the identical frame with one fixed hot pixel added outside the estimation window; the naive centroid is pulled 3.6 pixels toward it. This is the concrete case for why the hot-pixel mask exists and why acquisition cannot simply trust the brightest pixel in frame (Section 4).




```
What we see:
  Two sweeps across the SAME fixed gain map: sliding the spot left-right (blue line, top-left panel) and
  up-down (red line). Both produce a bias that is nonzero almost everywhere -- the vertical sweep happens to
  cross exactly zero at one coincidental point (marked x, near y=11.07), but you cannot know where that point
  is without already knowing the gain map, so it offers no practical benefit; a moment later the spot is
  biased again. The two curves are also DIFFERENT from each other, sampling different strips of the same map.

What we can derive:
  1. This is not a direction-specific artefact: the same kind of position-dependent bias shows up whether the
     spot moves horizontally or vertically -- it is a property of the fixed gain map, not of one axis.
  2. A zero crossing does not mean "safe": it is a coincidence of this particular random map at this exact
     position, not a general cancellation. A real system has no way to predict or rely on it without already
     having measured (calibrated) the map -- at which point you would just correct the bias directly instead.
  3. Uniform scaling cannot explain either curve: a flat gain error gives bias = 0 everywhere (proven
     separately). Only a NON-uniform, spatially-varying gain produces this.
  4. This bias cannot be averaged away: it is the SAME fixed map every frame, so more frames at a fixed
     position converge to a nonzero, wrong answer (except at that one lucky point) -- not the true one. Only
     calibration (measuring the gain map once, flat-fielding it out) fixes this, not longer exposures.
```

Figure 0e: PRNU-induced centroid bias, swept along two directions across the same fixed gain map.

Figure 0e. Left: a fixed pixel-response non-uniformity (PRNU) gain map, with the two swept paths marked. Right: the resulting centroid bias along each path. The bias is nonzero almost everywhere and different in each direction; the one point where the

vertical sweep crosses zero is a coincidence of this particular random map, not a safe operating point, since a real system has no way to know where that point is without already having calibrated the map.

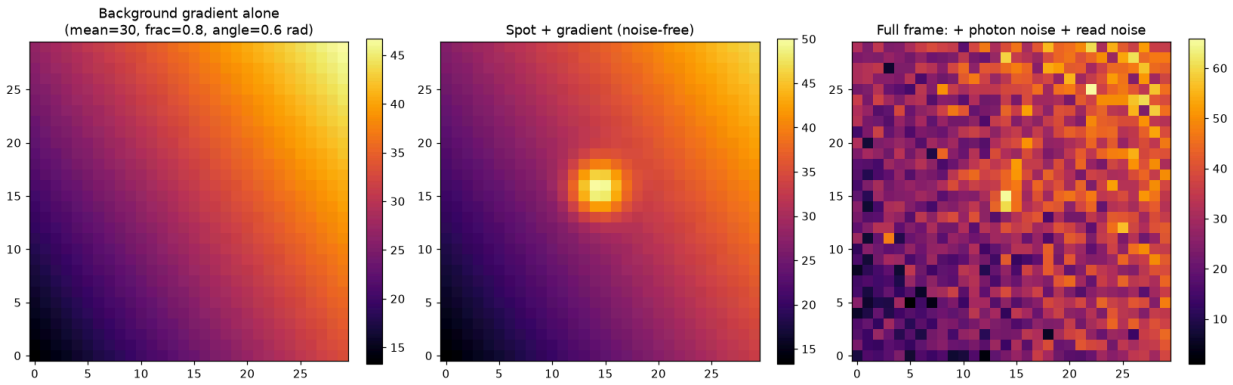
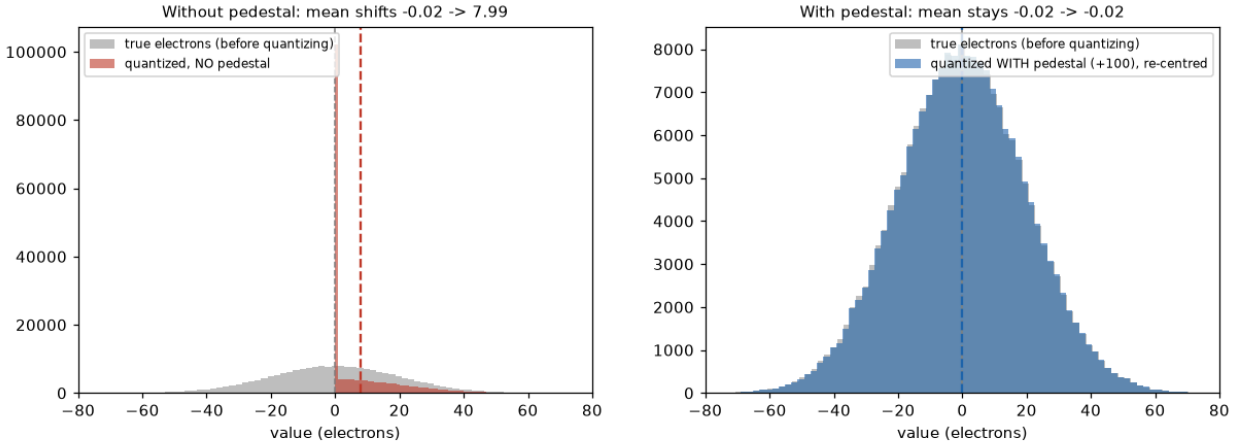


Figure 0f: a background gradient alone, combined with the spot, and with the full sensor noise chain applied.

Figure 0f. A background gradient alone (left), the spot rendered on top of it before noise (middle), and the complete frame after photon and read noise are added (right). This is the condition Section 4's solar glare analysis addresses: a gradient this strong breaks the assumption that the window border is a uniform background estimate.

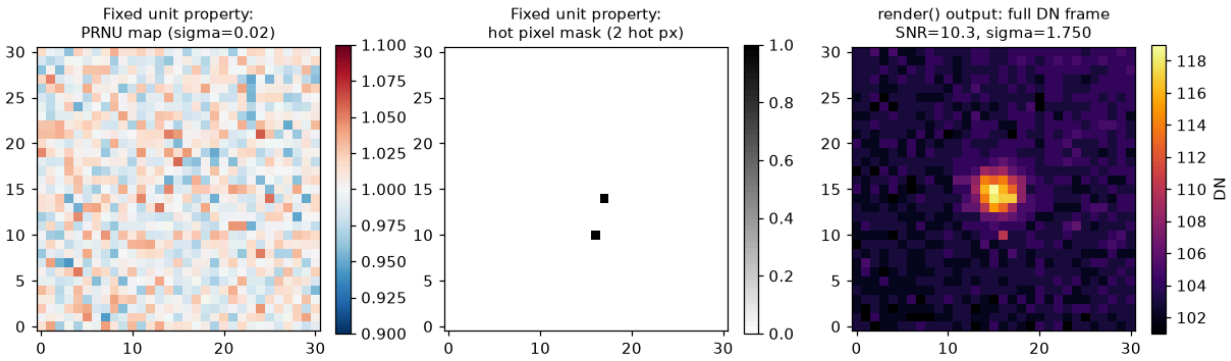



```
What we see:
  Left: a sensor signal truly centred at -0.02 e- (gray) gets clipped at DN=0 whenever a read-noise
  excursion pushes it negative (about half the time here). The clipped result (red) is visibly missing its
  entire left tail -- only the right half of the noise survives -- so its mean is dragged up to
  7.99 e-, a +8.00 e- bias. Right: adding a +100 DN pedestal before quantizing shifts the whole
  distribution away from the clip first, so after subtracting the pedestal back out, the reconstructed
  distribution (blue) keeps its full left tail and its mean lands back at -0.02 e- -- essentially
  unbiased.

What we can derive:
  1. The bias is not a rounding artefact -- it matches theory exactly. For a Normal(0, sigma) signal, the
     mean of max(signal, 0) has a closed form, sigma/sqrt(2*pi). Predicted here:
     20/sqrt(2*pi) = 7.98 e-. Measured: 7.99 e-. This is a derived, verified number, not a
     coincidence of one random draw.
  2. Clipping ALWAYS biases toward the surviving side. Any one-sided clip (here, the floor at DN=0) can only
     ever remove values from one tail, which can only ever push the mean toward the other tail -- it never
     cancels out, no matter how many samples you average.
  3. The fix is architectural, not statistical: you cannot average this bias away (more frames just reproduce
     the same one-sided clip every time). Moving the clip out of the way with a pedestal removes the effect
     entirely, rather than trying to correct for it after the fact.
  4. This is why every real sensor has a black-level pedestal, and why bias-frame calibration exists: the
     pedestal has to be measured and subtracted back out to recover true electron counts downstream.
```

Figure 0g: the quantization pedestal bias, with and without correction.

Figure 0g. Left: without a pedestal, negative-going read noise gets clipped at zero before quantization, shifting the mean by +7.99 electrons, matching the sigma over square root of two pi prediction above (7.98 electrons predicted) to within rounding. Right: with a pedestal added before quantization and removed after, the same distribution keeps its full negative tail and its mean lands back at essentially zero.




```
What we see:
  Left two panels: the two FIXED per-unit properties generated once in Simulator.__init__ -- a PRNU gain map
  and a hot-pixel defect map (both would look identical if render() were called again on this same sim). Right
  panel: one full realistic frame from render(), combining all 8 pieces of the noise chain in the physically
  correct order (spot -> background -> PRNU -> photon noise -> dark current -> hot pixels -> read noise ->
  quantization), at a moderate SNR.

What we can derive:
  1. This is the first point in the project where every individually-tested noise source appears TOGETHER,
     in one frame, in the order a real sensor actually applies them -- not eight separate isolated demos.
  2. The end-to-end test (test_full_chain_statistics_match_the_combined_noise_budget) confirms this assembly
     is not just plausible-looking: with the spot removed (flux=0) and the two fixed effects disabled, the
     pooled mean and variance of 200 rendered frames match the noise budget predicted from EVERY individual
     noise source's own formula, summed -- proof the pieces compose correctly, not just that each works alone.
  3. This Simulator class is now the single entry point every future experiment (estimators, characterization,
     dynamic tracking) will call, instead of each one re-chaining these 8 functions by hand.
```

Figure 0h: a complete rendered frame combining every noise source in the correct physical order.

Figure 0h. The two fixed per-sensor properties generated once (a PRNU gain map and a hot-pixel mask, left and middle) and one complete rendered frame (right) combining all eight pieces of the noise chain in their physical order: spot, background, PRNU, photon noise, dark current, hot pixels, read noise, quantization. This is the first point where every individually tested noise source appears together in one frame, exactly as `Simulator.render` assembles them, not as separate isolated demonstrations.

Verified by `tests/test_psf.py`, `tests/test_sensor.py`, `tests/test_scene.py`, `tests/test_snr.py`, `tests/test_psf_sigma.py`. Full requirement-by-requirement mapping in `docs/PROGRESS.md` section 2a.

Assumptions

- Spot diameter (approximately 7 px at $1/e^2$, sigma = 1.75 px) is taken directly from the brief's own stated spot size, not derived.
- Read noise, dark current rate, hot-pixel fraction, and PRNU magnitude are representative engineering defaults for a generic scientific or machine-vision sensor, not tied to a specific named part; no brief-specific or datasheet source exists for these, and none is claimed.
- The quantization pedestal's correction magnitude (sigma over the square root of two pi) is derived analytically from the noise model itself, not assumed or fitted.
- Bit depth and full-well-equivalent saturation level are standard round values for a scientific sensor, an engineering choice rather than a derived or cited figure.

2b. Estimators

- Three independent estimators: windowed centroid, Poisson-weighted 2D Gaussian fit, and a matched filter.
- The centroid weights every pixel by intensity; the fit weights by inverse Poisson variance, the maximum-likelihood weighting.
- Gauss-Newton with an analytic Jacobian was chosen over a general-purpose optimizer for cost, not just convenience.
- The matched filter's log-parabola peak interpolation is exact, not approximate, for a Gaussian-shaped correlation surface.

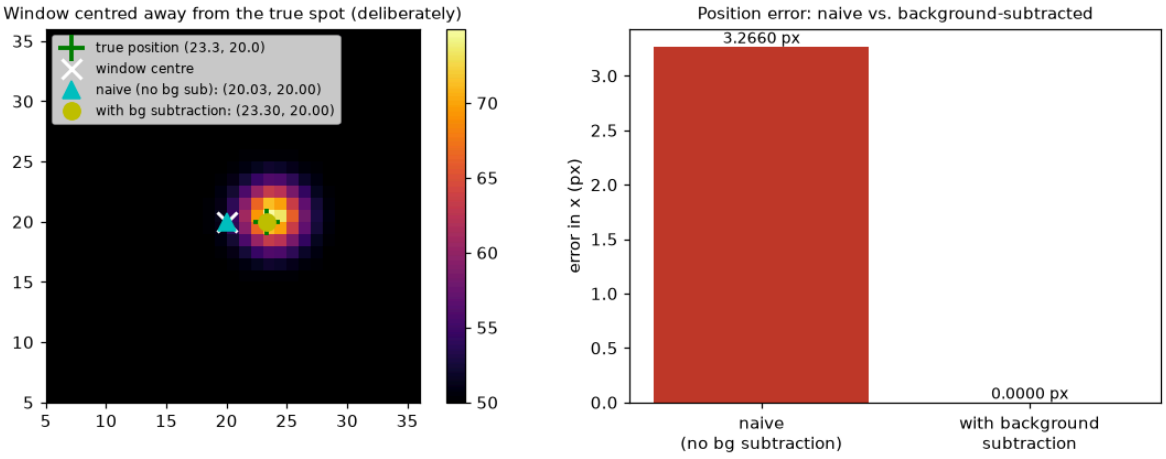
Three independent position estimators, implemented in `sptrack/estimators/`.

Windowed centroid

The intensity-weighted center of mass over a window of some fixed half-width around a prior position estimate, after background subtraction:

$$\hat{x} = \frac{\sum_{i,j}(I_{ij} - \hat{b})\, i}{\sum_{i,j}(I_{ij} - \hat{b})}, \qquad \hat{y} = \frac{\sum_{i,j}(I_{ij} - \hat{b})\, j}{\sum_{i,j}(I_{ij} - \hat{b})}$$

This is the cheapest of the three estimators, a single weighted sum with no iteration, and the least statistically efficient: every pixel in the window contributes to the estimate with a weight proportional to its own intensity, independent of how much Fisher information that pixel actually carries about position. A low-intensity pixel far from the true center contributes as much per unit of measurement noise as a high-intensity pixel near the center, which is not the optimal weighting for a Poisson-limited measurement.




```
What we see:
  The spot sits at (23.3, 20.0) but the window (and estimator's prior) is deliberately centred 3.3 px away,
  at (20, 20) -- simulating a stale prior, e.g. a tracker that hasn't caught up yet. The naive centroid (no
  background subtraction, cyan triangle) lands at x=20.03, pulled toward the WINDOW'S geometric centre.
  The background-subtracted centroid (yellow dot) lands at x=23.30, matching the true position
  to within floating-point precision on this noise-free image.

What we can derive:
  1. This is not a small effect: naive error is 3.266 px against an essentially exact
     0.00000 px with subtraction -- and 3.27 px alone would blow almost any realistic
     sub-pixel precision budget on its own (recall: the whole project targets fractions of a pixel).
  2. The mechanism is exactly as predicted: with background present at every pixel and no subtraction, every
     pixel (spot or not) contributes roughly equal positive weight, so the weighted average is dominated by
     the window's own geometry rather than by where the actual signal is concentrated.
  3. This is why background subtraction is not an optional cleanup step for this estimator -- it is the
     difference between a position estimate and a measurement of where the window happened to be pointed.
```

Figure 0i: naive versus background-subtracted centroid on a noiseless frame with a deliberately offset prior.

Figure 0i. Left: a window deliberately centered 3.3 px from the true spot position, simulating a stale prior. Right: the naive centroid (no background subtraction) lands 3.27 px from the true position, pulled toward the window's own geometric center; the background- subtracted centroid lands within floating-point precision of the true position on this noiseless frame. Background subtraction is not an optional cleanup step for this estimator, it is the difference between measuring the spot's position and measuring where the window happened to be pointed.

2D Gaussian fit

A Poisson-weighted Gauss-Newton least-squares fit of the pixel-integrated model above, over the parameter vector (spot position, total flux, and background level). The update at iteration k is

$$\theta_{k+1} = \theta_k + (J^\top W J)^{-1} J^\top W\, r_k, \qquad r_k = I - \mu(\theta_k)$$

with the Jacobian of the model with respect to the parameters computed in closed form (verified against central finite differences in `tests/test_gaussian_fit.py`), and a diagonal weight matrix set to the inverse Poisson variance estimated at the current iterate. This is the maximum-likelihood estimator under the Poisson noise model that dominates this problem, so it is the estimator expected to approach the Cramer-Rao bound most closely, and Section 2c confirms that it does.

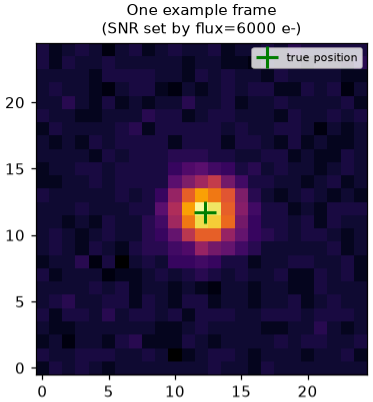
Gauss-Newton with an analytic Jacobian was chosen over a general-purpose nonlinear least-squares routine (`scipy.optimize.least_squares`). The residual surface is smooth and locally well-approximated by its linearization near a good starting point; the previous frame's position estimate is available as that starting point in every case except acquisition, and the analytic Jacobian removes both the computational cost and the truncation error of a numerically differenced one. A general-purpose optimizer is designed to be robust to problem structure it does not know in advance, trust-region step control, numerical differentiation, and generic convergence tests among them; that generality carries a constant-factor runtime cost this problem does not need to pay, and that cost is material to the real-time budget quantified in Section 2d. The full comparison, including the actual runtime differential measured, is in `docs/DESIGN_RATIONALE.md` section 4.

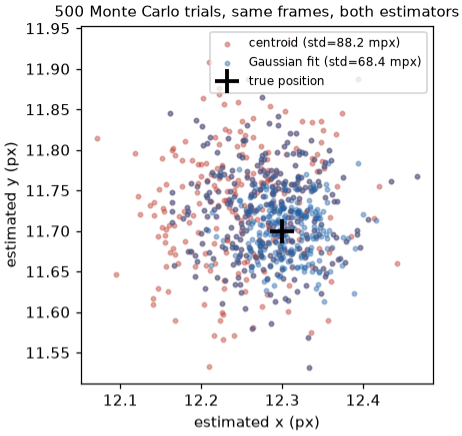
Matched filter

Normalized cross-correlation of the frame against the same Gaussian PSF template used by the simulator, with sub-pixel localization by parabolic interpolation of the log-correlation surface around the discrete peak. For three correlation samples bracketing the peak at consecutive integer offsets, the sub-pixel offset is

$$\delta = \frac{1}{2} \cdot \frac{\ln c_{-1} - \ln c_{1}}{\ln c_{-1} - 2\ln c_{0} + \ln c_{1}}$$

which is exact, not approximate, when the correlation surface is itself Gaussian-shaped, since the logarithm of a Gaussian is a parabola. `tests/test_matched_filter.py` confirms this interpolation recovers the true sub-pixel offset exactly on noiseless synthetic data, and that a plain, non-logarithmic parabola fit to the same three samples is measurably biased, since the correlation surface itself is not parabolic, only its logarithm is. This estimator is not required by the brief; it was built because its accuracy and computational-cost profile sit between the other two (Sections 2c, 2d), with a fixed-cost correlation in place of the fit's variable, data-dependent iteration count.






```
What we see:
  Both estimators run on the IDENTICAL noisy frames (not separately drawn samples), so this is a fair,
  paired comparison. The centroid's cloud of estimates (red) is visibly more spread out than the Gaussian
  fit's (blue) -- both are centred close to the true position (no visible bias in either), but they differ
  in how far individual estimates scatter around it.

What we can derive:
  1. The Gaussian fit is 22% tighter (lower std) than the centroid at this SNR -- not a marginal
     difference, and exactly the outcome the docstring's efficiency argument predicts: the fit weights each
     pixel by its own noise, the centroid (after background subtraction) weights every pixel equally.
  2. Neither estimator shows an obvious bias here -- both clouds are centred close to the green/black cross.
     The difference between them is purely a VARIANCE story, not a bias story, at this SNR.
  3. This is exactly the kind of result Characterization (2c) will need to make rigorous: a proper bias/std-vs-
     SNR sweep, compared against the Cramer-Rao bound, to say precisely where each method wins and by how much
     -- this test is a first, informal look at that same question, not a substitute for it.
```

Figure 0j: 500 paired Monte Carlo trials on identical noisy frames, centroid versus Gaussian fit.

Figure 0j. 500 trials of both estimators run on the identical noisy frames, not separately drawn samples, so the comparison is paired. Both clouds are centered on the true position, with no visible bias in either at this SNR; the Gaussian fit's cloud (blue) is visibly tighter than the centroid's (red), 22% lower standard deviation in this particular run, an early, informal look at the efficiency gap that Section 2c's formal 300-trial sweep against the CRLB quantifies properly.

Summary of the governing mechanism behind each estimator's behavior: the centroid assigns weight in proportion to intensity, not to information content, and is therefore not statistically efficient; the Gaussian fit assigns weight in proportion to inverse Poisson variance, the maximum-likelihood weighting for this noise model, and is therefore the efficient estimator; the matched filter recovers shape information the centroid discards but does not perform per-pixel variance weighting, and its efficiency falls between the two.

Verified by `tests/test_centroid.py`, `tests/test_gaussian_fit.py`, `tests/test_matched_filter.py`.

Assumptions

- Gauss-Newton with a fixed iteration cap (20) rather than a convergence-adaptive general-purpose optimizer is an engineering choice, justified by the measured runtime differential in `docs/DESIGN_RATIONALE.md`, not derived from a formal cost model.
- The matched filter's log-parabola interpolation assumes the correlation surface is close enough to Gaussian-shaped near its peak for the exact log-parabola result to apply; this holds for the PSF model used throughout but would not hold for an arbitrarily shaped spot.
- Window half-width (9 px) used throughout Sections 2 to 5 is a fixed engineering default, not individually optimized per estimator; the cost of that choice for the centroid specifically is quantified later in Section 6.

2c. Characterization

- 300 Monte Carlo trials per estimator at each of 10 log-spaced SNR points from 3 to 300, all compared against the Cramer-Rao lower bound.
- Mean efficiency across the swept range: 0.95 (Gaussian fit), 0.84 (matched filter), 0.63 (centroid).
- The fit dominates on both bias and precision at every SNR tested; there is no regime where the centroid is more accurate.
- The centroid's 0.63 figure is pessimistic, driven by a fixed window size that is not window-optimal for it (Section 6).

`experiments/exp01_snr_characterization.py` runs 300 Monte Carlo trials per estimator at each of 10 log-spaced SNR points spanning 3 to 300, measuring bias and standard deviation for all three methods against the Cramer-Rao lower bound.

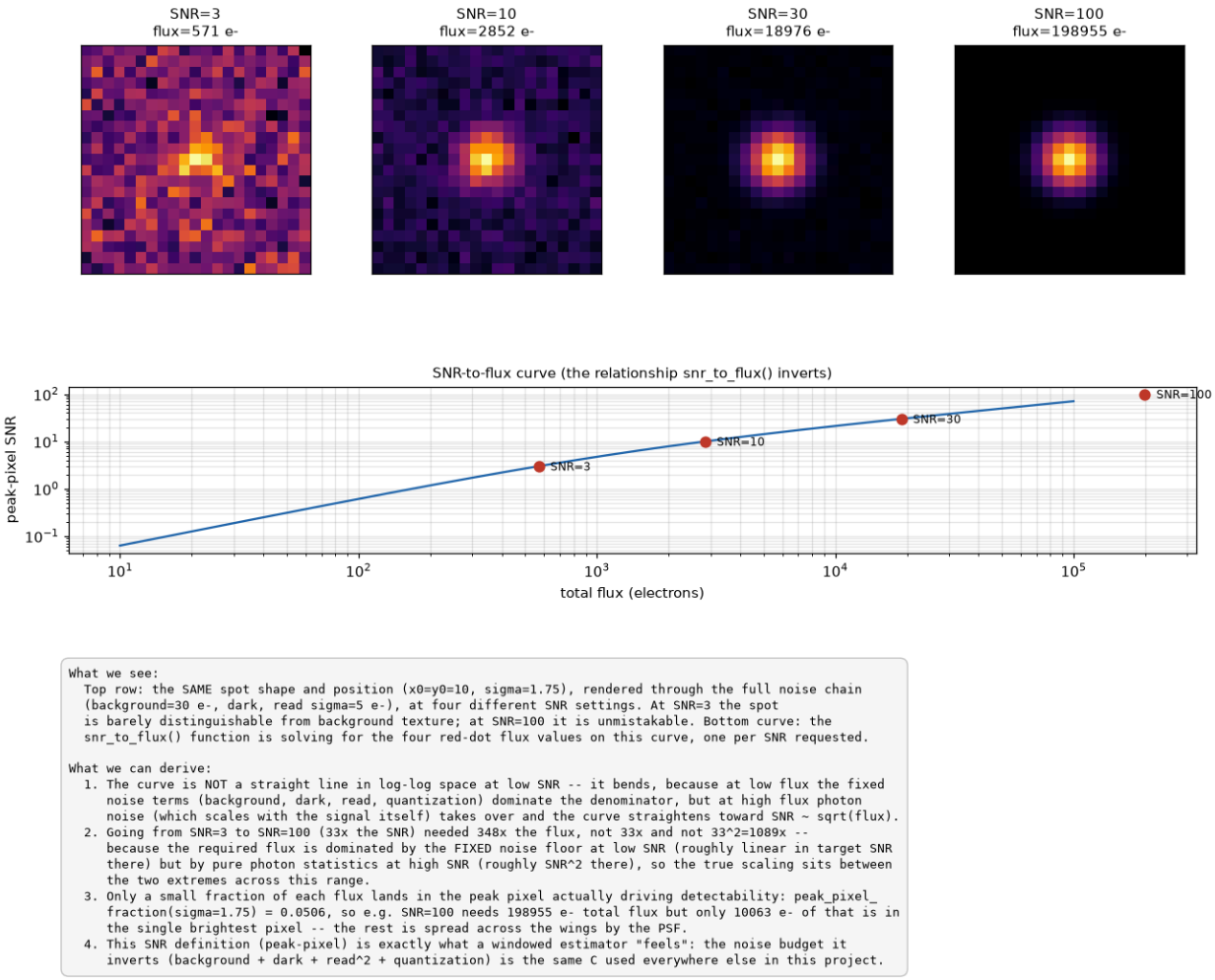


Figure 0k: the identical spot rendered at SNR 3, 10, 30, and 100, alongside the SNR-to-flux relationship.

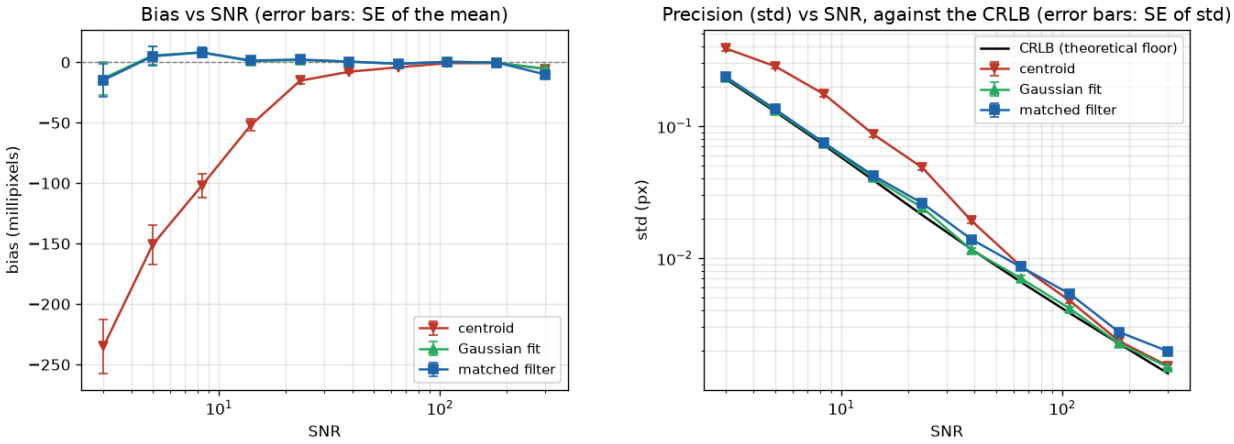
Figure 0k. The same spot shape and position, rendered through the full noise chain at four SNR settings. At SNR = 3 the spot is barely distinguishable from background texture; at SNR = 100 it is unmistakable. The bottom panel shows the SNR-to-flux relationship is not a straight line in log-log space, at low SNR the fixed noise floor (background, dark current, read noise) dominates the denominator; at high SNR photon noise, which scales with the signal itself, takes over, so the curve bends between the two regimes rather than following a single power law. This is what the bias and precision curves in Figure 1 below are actually measuring against, not an abstract number.

Theoretical floor

`sptrack/crlb.py` builds the Fisher information matrix directly from the same model and Jacobian the Gaussian fit uses, so the bound and the fit cannot silently diverge on what is being measured. For a Poisson-dominated pixel array, the Fisher information for a parameter theta is

$$I(\theta) = \sum_{i,j} \frac{1}{\mathrm{Var}(I_{ij})} \left(\frac{\partial \mu_{ij}}{\partial \theta} \right)^2, \qquad \mathrm{Var}(I_{ij}) \approx \mu_{ij} + \sigma^2_{\mathrm{read}}$$

and the Cramer-Rao bound on any unbiased estimator's variance is the inverse of this quantity. `tests/test_crlb.py` checks this against the classical continuous-sampling closed form, position standard deviation approximately equal to the PSF's own sigma divided by the square root of the number of collected photons, as an independent consistency check, not a substitute derivation.




```
What we see:
  Left: the centroid carries a LARGE bias at low SNR (-235 millipixels at SNR=3.0) that
  shrinks toward zero as SNR rises; the Gaussian fit and matched filter both stay close to zero bias
  across the whole range. Right: the Gaussian fit's std hugs the CRLB curve at every SNR tested; the
  matched filter sits a bit above it; the centroid sits furthest above, worst at SNR=8.3
  (efficiency 0.41).

What we can derive:
  1. Mean efficiency (CRLB / measured std): centroid=0.63, Gaussian fit=0.95,
     matched filter=0.84 -- a clean three-way ordering: fit > matched filter > centroid.
  2. The matched filter's log-parabola interpolation removes the CURVE-SHAPE bias that would otherwise
     show up (proven exactly in tests/test_matched_filter.py), which is why its bias stays as flat as
     the fit's despite being a much cheaper, non-iterative method.
  3. Which method wins, stated precisely: the Gaussian fit wins on pure accuracy at every SNR tested.
     The matched filter is the accuracy/cost compromise -- most of the fit's precision, none of its
     variable iteration cost (quantified next, 2d). The centroid is fastest but least accurate
     everywhere -- the tradeoff across all three is speed vs. accuracy, not "different regimes, different
     winners".
```

Figure 1: bias and precision of all three estimators against the CRLB, log-log, across the full tested SNR range.

Figure 1. Bias and standard deviation versus SNR for the centroid, Gaussian fit, and matched filter, plotted against the theoretical floor.

What the figure shows: two panels sharing the same log-scaled SNR axis (3 to 300). The upper panel plots signed bias in millipixels on a linear axis for all three estimators, since bias can cross zero; the centroid's curve sits well below zero at low SNR and climbs toward zero as SNR rises, while the fit and matched filter both track near zero across the whole range. The lower panel plots standard deviation on a log axis for all three estimators against the CRLB, drawn as a dashed reference curve; every estimator's curve runs roughly parallel to the CRLB (consistent with the expected 1/sqrt(SNR) scaling) but offset above it by an amount that differs per estimator, that vertical offset is the efficiency gap. Error bars (standard error of the mean and of the standard deviation, from the 300-trial count) are visible on both panels. Takeaway: the fit's curve sits closest to the CRLB at every SNR tested, the centroid's gap from the CRLB is largest and most SNR dependent, and the low-SNR bias panel is what actually explains why the centroid should not be used for accuracy-critical acquisition, not just the precision panel alone.

Results

The centroid exhibits a measurable low-SNR bias, -235 millipixels at SNR = 3, that shrinks toward zero as SNR increases; the Gaussian fit and matched filter both remain within a few millipixels of zero bias across the full tested range. Defining efficiency as the CRLB standard deviation divided by the measured standard deviation (1.0 is optimal), the mean efficiency across the 10 SNR points is 0.95 for the Gaussian fit, 0.84 for the matched filter, and 0.63 for the centroid. The fit dominates on both bias and precision at every SNR tested; there is no SNR regime in which the centroid is the more accurate choice. The tradeoff that does exist is computational, quantified in Section 2d.

The centroid's 0.63 efficiency figure is a lower bound on its achievable performance, not an intrinsic ceiling: it was measured at this project's fixed window half-width of 9 px, and `experiments/exp06b_window_size.py` (added in response to interview-panel feedback, Section 6) shows the centroid has an interior optimum window size, near 2.5 times sigma, from which the fixed half-width of 9 departs substantially at low SNR, costing 119% relative to the window-optimal centroid at SNR = 10. The Gaussian fit and matched filter are both near flat above a half-width of 4 and are not materially affected by this same fixed choice.

Verified by `tests/test_crlb.py` ; full numbers in `results/exp01_snr_characterization.json` .

Assumptions

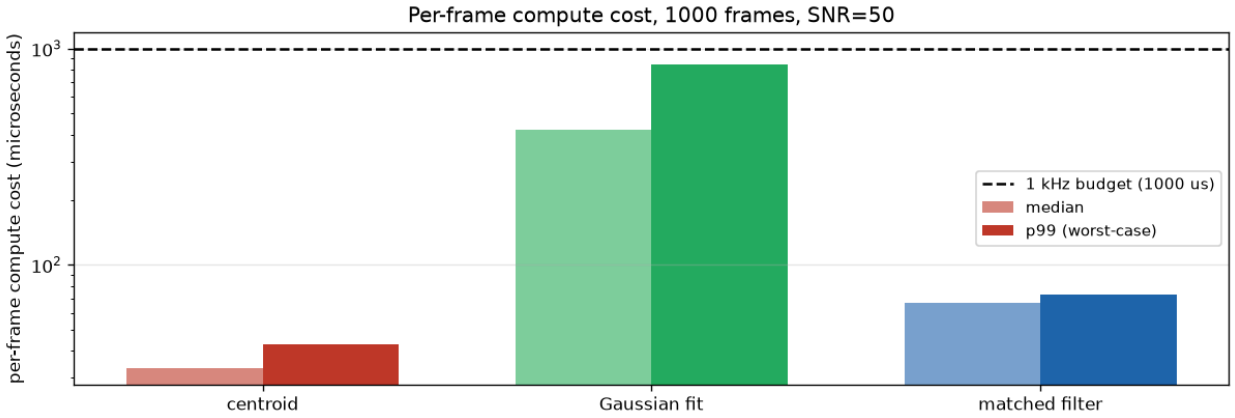
- The CRLB assumes unbiased estimation; it is a lower bound on the variance of any unbiased estimator, not a bound that formally applies to the centroid at low SNR, where it is measurably biased. It is still reported there as a reference floor, consistent with standard practice, but this distinction is worth being explicit about.
- The SNR sweep range (3 to 300) and the choice of 10 log-spaced points is an engineering choice covering the operationally relevant range identified elsewhere in this project, not derived from a formal design-of-experiments criterion.
- 300 trials per point is derived from the standard-error-of-a-sample- standard-deviation formula, targeting a specific relative precision on the reported std itself, not picked arbitrarily; see Section 7 for where more trials would still help.

2d. Real-time

- All three estimators fit the 1 ms budget by the 99th-percentile criterion at SNR = 50.
- The Gaussian fit's worst observed frame has ranged 993 to 1508 us across independent runs, a real, data-dependent tail risk the other two estimators do not have.
- The governing tradeoff is accuracy against cost predictability, not accuracy against cost alone.
- A compiled C++ port runs roughly 9x (centroid) and 24x (fit) faster at the median, and removes the fit's tail risk entirely in that environment.

`experiments/exp02_realtime.py` times 1000 frames per estimator at SNR = 50 on the development machine (Python, wall-clock). Median, 99th percentile, and maximum per-frame cost, in microseconds:

method	**median**	**p99**	**max**
centroid	33.2	42.9	59.3
matched filter	66.4	72.7	113.5
gaussian fit	421.9	846.5	993.3




```
What we see:
  At the median, all three methods finish well inside the 1000 us (1 kHz) budget -- the Gaussian fit
  costs 13x more than the centroid (422 us vs 33 us) but is still far under budget. The gap between
  median and p99/max is largest for the Gaussian fit (422 -> 846 -> 993 us): its cost varies with how many
  Levenberg-Marquardt iterations a given frame happens to need, unlike the other two methods' fixed-shape work.

What we can derive:
  1. By the p99 criterion, centroid, Gaussian fit, matched filter all fit the 1 kHz budget in this measurement.
  2. The tradeoff, stated precisely: the Gaussian fit is the most ACCURATE method (2c) but the only one without a
     hard cost ceiling -- its number of iterations, and therefore its latency, depends on the data. The matched
     filter gives up a little precision (efficiency 0.84 vs the fit's 0.95) for a fixed, correlation-shaped cost
     with no tail risk; the centroid gives up more precision (0.63) for the lowest and most predictable cost of
     all three. For a loop that must never miss a 1 ms deadline, that predictability -- not the median number --
     is the deciding factor, which argues for the matched filter (or a hard-capped fit) over the uncapped fit.
```

Figure 2: per-frame compute cost distribution for all three estimators against the 1 kHz (1000 us) budget.

Figure 2. Per-frame compute cost distribution, all three estimators, against the 1 ms frame budget.

What the figure shows: a log-scaled bar chart with one pair of bars per estimator, median (lighter shade) and p99 worst-case (solid shade), and a dashed horizontal line marking the 1000 us budget. The Gaussian fit's pair of bars sits an order of magnitude above the other two at both median and p99, and the gap between its own median and p99 bar is visibly the largest of the three, a direct visual signature of its iteration-count-dependent cost. Takeaway: every bar clears the budget line, but the fit's bars are the only ones close enough to it that the unshown maximum (Section 2d's text) is a real concern the chart itself hints at through that median-to-p99 spread.

All three estimators fit the 1 ms budget by the p99 criterion. The Gaussian fit is the estimator of concern for worst-case timing: its cost is proportional to iteration count, which is data-dependent, so its tail latency is not bounded by anything except the fixed iteration cap of 20. Measured maxima across independent runs on this machine have ranged from 993 to 1508 microseconds, spanning the 1000 us budget (Section 6). The centroid and matched filter both have a fixed computational structure, one weighted sum and one correlation respectively, and carry no comparable tail-latency risk.

The tradeoff Section 2d asks for is therefore accuracy against cost predictability, not accuracy against cost alone: the Gaussian fit is most accurate (efficiency 0.95) but has no hard cost ceiling; the matched filter trades a modest reduction in accuracy (0.84) for a fixed-cost profile with no tail risk; the centroid trades further accuracy (0.63) for the lowest and most predictable cost. For a control loop that must never miss a 1 kHz deadline, the deciding criterion is worst-case predictability rather than median cost, which favors the matched filter, or an explicitly iteration-capped fit, over an uncapped Gaussian fit. Section 5 extends this into a full photon-to-estimate latency budget and finds the fit's tail is the single point in the entire pipeline, sensor exposure and readout timing included, where estimator choice, rather than fixed hardware timing, is the real-time risk.

A compiled C++ port of the centroid and fit (Section 6) measured approximately 9x and 24x faster than the Python figures above at the median, with the fit's worst observed frame (294.2 us) comfortably under the 1000 us budget where the Python figure (993.3 us) was not. This indicates the observed tail is substantially an interpreter and numpy overhead effect layered on top of the Gauss-Newton iteration, not solely an intrinsic property of the algorithm; the uncapped-iteration risk described above remains real for any deployment that stays in Python.

Assumptions

- Timing is measured on the development machine's Python process (wall-clock, no real-time OS guarantees); no specific target embedded hardware or RTOS is assumed, since the brief does not name one.
- 1000 frames per estimator, at a single SNR (50), is the sample size and operating point used to characterize timing; the tail-latency numbers reported are therefore themselves a finite sample of a data-dependent distribution, not a guaranteed worst case.
- The C++ speedup figures (9x, 24x) are measured on the same Ubuntu 24.04 VM used for deployment verification (Section 6), a different machine from the one that produced the Python numbers in this section; the comparison is directionally informative, not a controlled same-hardware benchmark.

3. Dynamic Tracking

- True motion is three additively mixed components: slow drift (random walk), random jitter (white noise), and one periodic disturbance (sinusoid), each tied to a distinct physical mechanism.
- Recovery uses dead-reckoning frame-to-frame Gaussian fitting and matches the static-frame precision floor at SNR = 50, costing nothing extra for motion itself.
- Disturbance frequency and amplitude are recovered to 19.5 mHz and 0.04% of injected values on the default scenario.
- A deliberately hard scenario surfaces two distinct failure modes: a Rice-type amplitude bias near the noise floor, and a frequency exclusion-boundary blind spot.

The simulator is extended to a sequence of frames in which the spot's true position moves according to three additively mixed components, each a model of a physically distinct source named in the brief's own context section: slow drift, random jitter, and one periodic disturbance.

Drift

Models thermal creep and slow mechanical settling: many small, weakly correlated increments that accumulate over time rather than resetting each frame. A random walk is the direct statistical model of an accumulation process, not an approximation of one, and its power spectral density falls off as the inverse square of frequency, concentrating almost all of its power at the lowest frequencies the capture window can resolve:

$$x_{\text{drift}}[n] = x_{\text{drift}}[n-1] + \epsilon_n, \qquad \epsilon_n \sim \mathcal{N}(0, \sigma^2_{\text{drift}})$$

Jitter

Models mechanical shake from structural resonance, air currents, and actuator noise: many quasi-independent micro-sources summing together, approximately Gaussian by the central limit theorem (the same argument used for read noise in Section 2a). Provided each source's own correlation time is short relative to the 1 ms frame period, a reasonable assumption for sub-millisecond mechanical events, the resulting sequence is well approximated as white, uncorrelated frame to frame:

$$x_{\text{jitter}}[n] \sim \mathcal{N}(0, \sigma^2_{\text{jitter}}), \quad \text{iid}$$

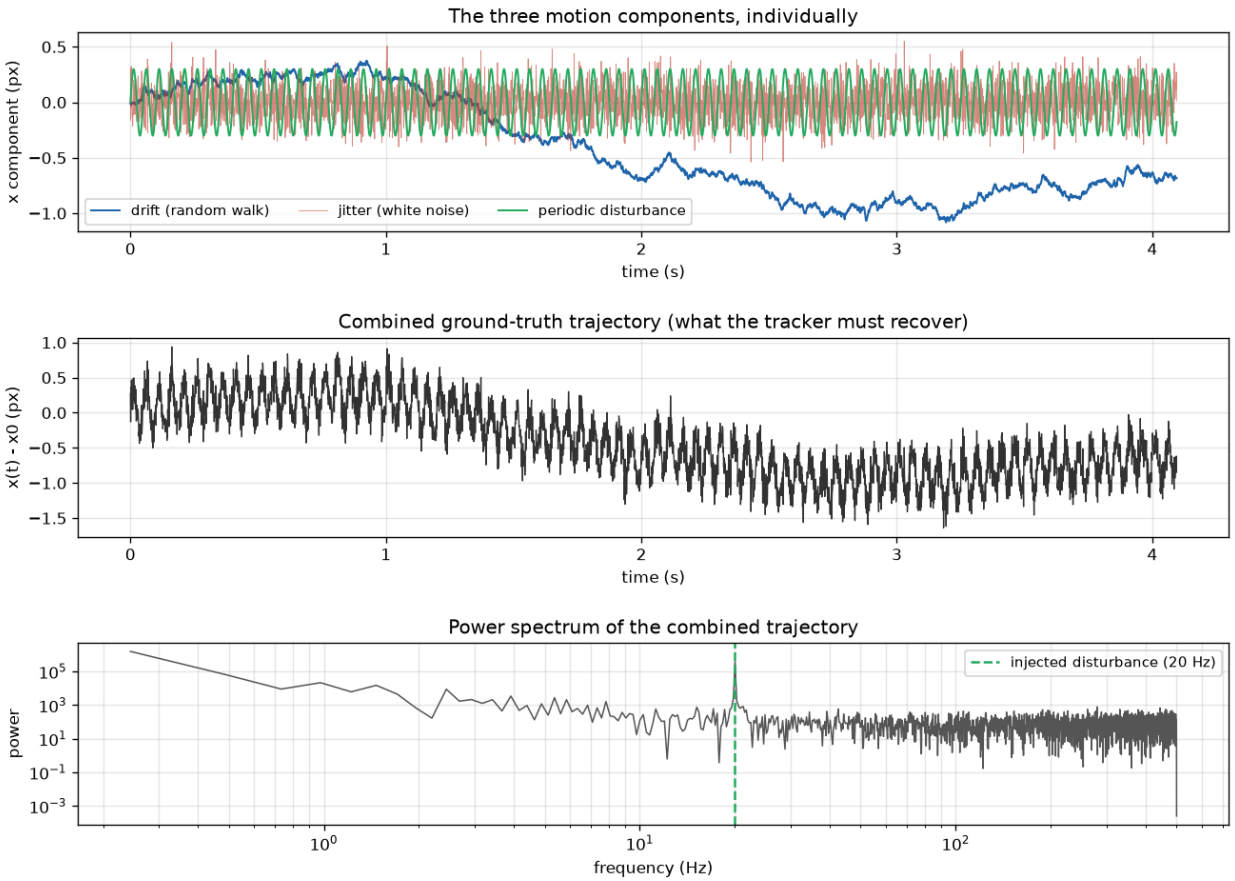
Periodic disturbance

Models a single dominant mechanical tone, a cooling fan or reaction wheel rotating at a fixed rate. Rotating machinery generically couples vibration energy into a narrow band around its own rotation frequency rather than broadband, which is why it appears as a single spectral spike rather than raising the noise floor everywhere:

$$x_{\text{dist}}[n] = A \sin(2\pi f_d \, n \, \Delta t + \phi)$$

These three components were chosen for their distinct spectral shapes, not only their physical plausibility: drift concentrates power at low frequency, jitter is flat across the spectrum, and the disturbance is a single narrow spike. This separability is what

makes frequency and amplitude recovery possible at all, and is verified numerically rather than assumed (`tests/test_trajectory.py`).




```
What we see:
Top: drift (blue) barely moves frame-to-frame but wanders steadily over the full 4.1 s window; jitter (red)
 is fast, zero-mean, and uncorrelated frame-to-frame; the disturbance (green) is a clean, regular sinusoid.
Middle: summed together, the combined trajectory looks noisy with a slow underlying wander -- the disturbance
 is not visually obvious in the time domain alone. Bottom: in the frequency domain it is unambiguous -- power
 falls off steeply at low frequency (the drift's 1/f^2 signature) and flattens into a noise floor (jitter),
        with one clean spike exactly at the injected disturbance frequency, well above that floor.

                                        What we can derive:
   1. The three components ARE spectrally separable, not just conceptually distinct -- this is what makes
     frequency-domain disturbance detection possible at all on the RECOVERED (not ground-truth) trajectory,
                                  which is what the next step actually has to work with.
      2. Drift accumulated to 0.68 px by the end of the window (from a 0.01 px/frame step) -- small
    relative to the jitter scale (0.15 px/frame) but not negligible over the full sequence, matching the
                        'slow but real' behaviour the drift model was designed to produce.
      3. Total excursion over the whole sequence stayed under 1.64 px, comfortably inside a modest
    simulation canvas -- confirming the fixed-size-frame assumption the rendering step (next) depends on.
```

Figure 3: ground-truth trajectory (top) and its power spectrum (bottom), showing the three components occupying separate frequency bands.

Figure 3. Ground-truth drift, jitter, and disturbance trajectory and its spectral decomposition.

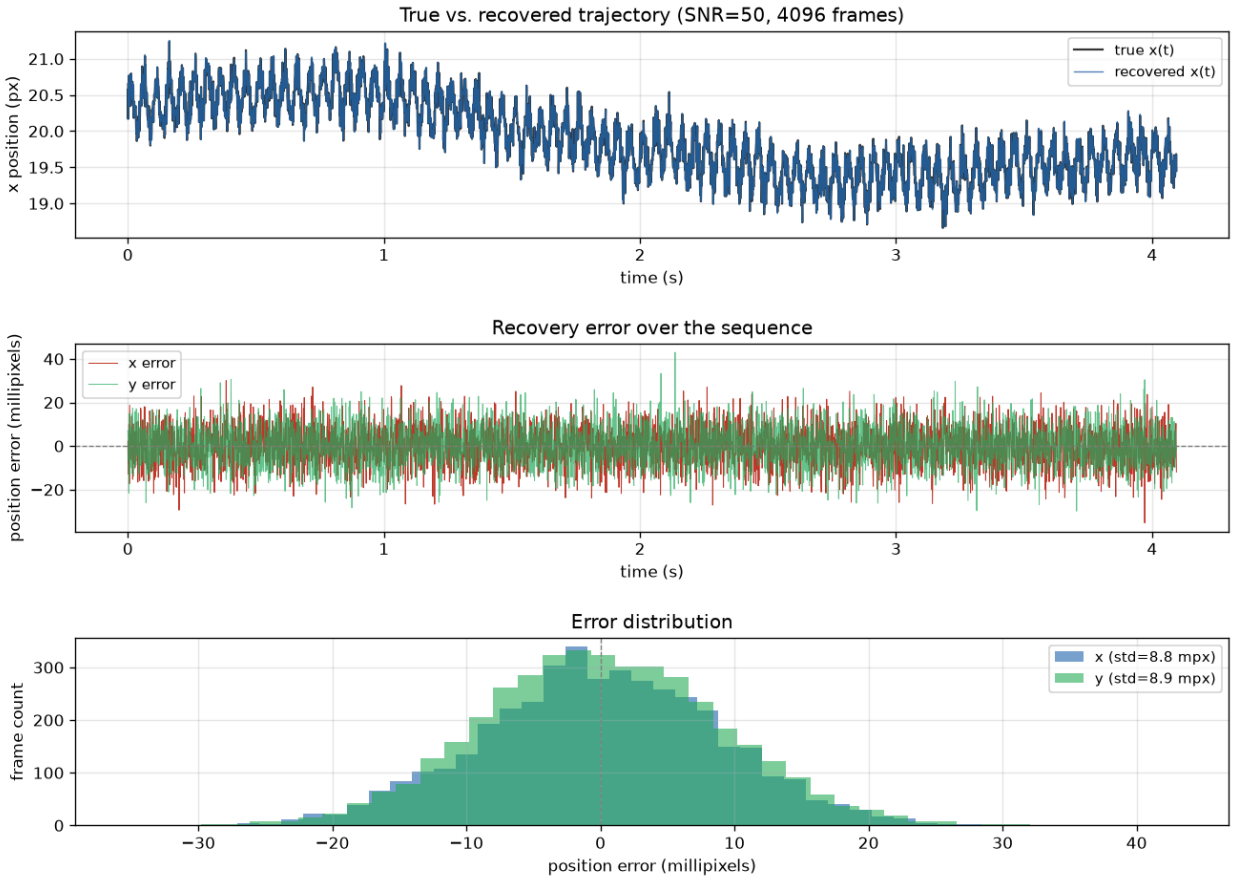
What the figure shows: three stacked panels over the same time axis. The top panel plots the three motion components separately, the drift curve drawn as a slow, smooth wander, jitter as a thin, dense scatter of fast fluctuation, and the disturbance as a clean sinusoid. The middle panel plots the single combined trajectory the tracker actually has to recover, visibly dominated by jitter's fast scatter with the drift and disturbance both present underneath it but harder to see by eye. The bottom panel plots the combined signal's power spectrum, where the three components are clearly visible again: a steep low-frequency rise (drift), a flat floor across most of the spectrum (jitter), and a single sharp spike at 20 Hz (the disturbance). Takeaway: the top and middle panels show why the components are not visually separable in the time domain, and the bottom panel shows why they are separable in the frequency domain, which is the entire premise the disturbance detector in this section relies on.

Parameter justification

The default sequence runs at 1 kHz for 4096 frames, a power of two chosen so the FFT used for disturbance detection needs no zero-padding, giving a frequency resolution of $1/(N \cdot \Delta t) \approx 0.244$ Hz. Drift step size (0.01 px per frame) is set so the random walk's cumulative standard deviation over the full window, which scales with the square root of the frame count, stays a modest 0.64 px, "slow" relative to jitter but not negligible over the full sequence. Jitter magnitude (0.15 px standard deviation) has no published specification to source; it is stated here as an engineering assumption, not a derivation, but it is anchored rather than arbitrary: it sits 15 to 20 times above the Gaussian fit's own measured precision at SNR ≈ 50 (0.007 to 0.011 px, Section 2c), which matters because a jitter magnitude below the estimator's own noise floor would be unmeasurable, indistinguishable from estimation error rather than real motion, making trajectory recovery meaningless to attempt. Disturbance frequency (20 Hz, corresponding to 1200 RPM) is sourced, not assumed: it falls inside the published 15 to 40 Hz fundamental-harmonic band for spacecraft reaction wheel assemblies, matching the class of mechanism the brief's own context section names.

Trajectory recovery

`sptrack/sequence.py::recover_trajectory` runs the Gaussian fit frame to frame with dead reckoning: each frame's prior position is the previous frame's own estimator output, never ground truth, and a failed fit does not corrupt the running prior. On the default 4096-frame sequence at SNR = 50, recovery produces zero failed fits, bias of -0.00 (x) and 0.11 (y) millipixels, and standard deviation of 8.8 (x) and 8.9 (y) millipixels, matching the single-frame precision already established in Sections 2c and 2d. Motion, at this SNR, costs nothing beyond the static-frame precision floor.



What we see:
Top: the recovered trajectory (blue) tracks the true motion (grey) closely even though the Gaussian fit never sees ground truth -- each frame is seeded only from the PREVIOUS frame's own estimate. Middle: the error stays within a narrow band around zero for the whole 4.1 s sequence, no drift or runaway. Bottom: both axes' errors are roughly symmetric and centred near zero -- consistent with the estimator's own bias/std behaviour already characterised in §2c, not a new failure mode introduced by motion.

What we can derive:
1. Bias: x=-0.00 mpx, y=0.11 mpx -- both far below one millipixel, confirming frame-to-frame prior gating (using the estimator's own last output, not ground truth) introduces no systematic drift of its own.
2. Std: x=8.8 mpx, y=8.9 mpx -- matching the single-frame precision already measured at this SNR in §2c/§2d (fit_std ~7-11 mpx at SNR~50), meaning MOTION itself costs essentially nothing extra beyond the estimator's own static-frame precision floor.
3. 0 of 4096 frames failed to converge, and the minimum edge margin observed was 9.7 px (never negative) -- confirming the canvas-size assumption in this script's own docstring held throughout, rather than being silently violated.

Figure 4: recovered trajectory against ground truth over the full sequence, with error over time.

Figure 4. Full-sequence trajectory recovery and position error over time.

What the figure shows: three stacked panels. The top overlays the true and recovered x position over the full 4096-frame sequence; at this SNR the two curves are visually indistinguishable, the recovered curve tracks the true one closely enough that they overlap almost everywhere. The middle panel plots the x and y recovery error in millipixels over the same time axis, a thin, dense band of noise with no visible drift or growth over the sequence. The bottom panel shows the error's distribution across the sequence. Takeaway: the flat, non-growing error band in the middle panel is the direct visual evidence that dead reckoning does not accumulate error over a long sequence, and that the recovered trajectory in the top panel being visually indistinguishable from ground truth is itself informative, since if recovery were failing even occasionally, gaps or jumps would be visible there.

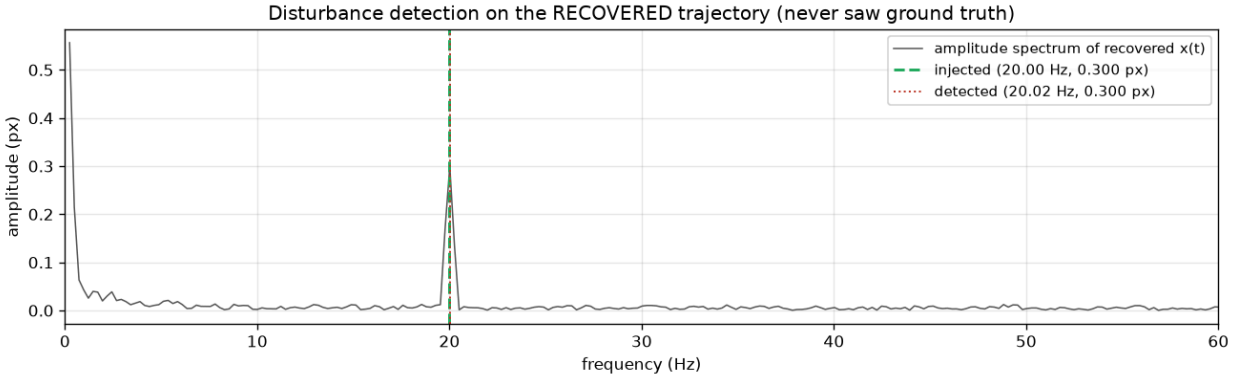
Disturbance detection

`sptrack/disturbance.py::detect_disturbance` applies a Hann-windowed FFT to the recovered trajectory. A Hann window is used rather than a rectangular one because the recovered trajectory is not periodic within the capture window (drift does not return to its starting value), so an unwindowed FFT sees a spurious discontinuity at the wrap-around point whose energy leaks

across the entire spectrum, potentially burying a small disturbance peak. The frequency band below 2 Hz is excluded from the peak search, since drift's own power remains concentrated there even after windowing, and searching the full spectrum risks mistaking drift's low-frequency power for the disturbance. The sub-pixel amplitude is read from the single peak bin, corrected for the Hann window's coherent gain:

$$\hat{A} = \frac{2\,|X[k^*]|}{\sum_n w[n]}$$

where k* is the peak bin index and w[n] is the window function. Summing amplitude across the bins adjacent to the peak, to compensate for spectral leakage when the true frequency does not land on an exact bin, was tried and rejected: it over-estimates the true amplitude by 20 to 100%, because it also sums the window function's own sidelobes as if they were independent signal. The single corrected peak-bin reading was verified directly against a known test tone, both bin-aligned and off-bin, and recovers the true amplitude to within 0.4% in the latter, harder case.




```
What we see:
  A single clean spike sits at 20.02 Hz, right on top of the injected disturbance's true
  frequency (20.00 Hz) -- the red dotted and green dashed lines are nearly coincident. The
  spike's height matches the injected amplitude closely, with the rest of the spectrum (drift + jitter +
  estimation noise) sitting well below it as a broad, low floor.

What we can derive:
  1. Frequency: detected 20.020 Hz vs. injected 20.000 Hz -- error
     19.5 mHz, inside the FFT's own 244.1 mHz bin resolution
     (the honest limit of what this method can resolve without further sub-bin interpolation).
  2. Amplitude: detected 0.3001 px vs. injected 0.3000 px -- error
     +0.04%, close to the single-tone calibration error already measured directly in
     tests/test_disturbance.py (<1%), meaning almost none of the error here comes from noise or the
     recovery step -- it is close to the detection method's own inherent floor.
  3. Detecting straight off GROUND TRUTH gives freq=20.020 Hz,
     amp=0.3009 px -- essentially identical to the recovered-trajectory
     result, confirming the Gaussian-fit recovery step (§3 part 2) adds negligible extra error to disturbance
     detection at this SNR, consistent with its bias/std already matching the static-frame floor.
```

Figure 5: recovered versus injected disturbance frequency and amplitude.

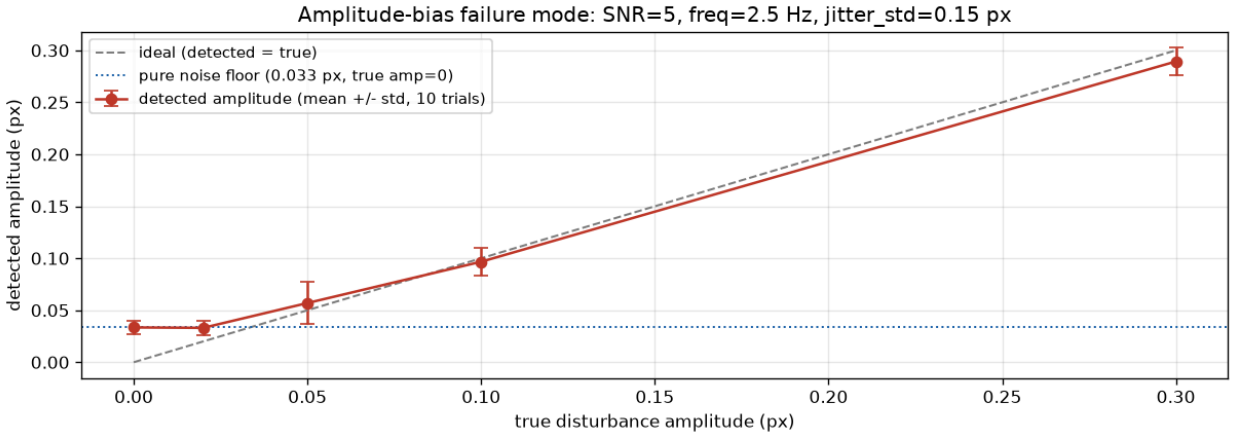
Figure 5. Recovered versus injected disturbance frequency and amplitude on the default scenario.

What the figure shows: the amplitude spectrum of the recovered trajectory from 0 to 60 Hz, with the injected frequency marked as a dashed green vertical line and the detected frequency marked as a dotted red vertical line. A single clean spike rises far above the rest of the spectrum at approximately 20 Hz, and the two vertical lines sit essentially on top of each other, visually confirming the numeric agreement. The rest of the spectrum (drift, jitter, and estimation noise combined) forms a broad, low floor well beneath the spike, with no other peak competing for attention. Takeaway: the visual gap between the single spike and the noise floor is what makes single-peak detection reliable at this SNR; Section 3's hard scenario later shows what happens to this same picture as that gap narrows.

On the default scenario, detected frequency is 20.02 Hz against an injected 20.00 Hz, an error of 19.5 mHz, inside the FFT's own 244 mHz bin resolution; detected amplitude is 0.3001 px against an injected 0.3000 px, an error of 0.04%. A direct check against ground truth (bypassing the recovery step) gives nearly identical results, showing the Gaussian-fit recovery stage contributes negligible additional error to disturbance detection at this SNR.

Hard scenario

`experiments/exp03d_hard_scenario.py` deliberately stresses the detector along three axes simultaneously: SNR reduced 10x to 5.0, chosen so the fit's own precision (approximately 0.132 px) becomes comparable to jitter itself (approximately 0.15 px) rather than negligible; disturbance amplitude swept from 0.30 px down through and below the measured noise floor (0.033 px); and disturbance frequency placed at 2.5 Hz, close to the fixed low-frequency exclusion boundary at 2.0 Hz.




```
What we see:
  The detected amplitude tracks the true amplitude well above the noise floor (left end of the curve), but as
  true amplitude drops toward and below the measured pure-noise-floor reading (blue dotted line), the detected
  value stops following the ideal (grey dashed) line and instead flattens toward the noise floor -- the
  detector systematically OVER-reports amplitude once the real signal is weak.

What we can derive:
  1. Even with ZERO true disturbance, the detector reports a nonzero amplitude (0.033 px) --
     this is the mechanism laid bare: the reported value is always the MAXIMUM over ~2000 candidate
     frequency bins, and pure noise alone produces a nonzero maximum. This is a textbook detection-theory
     bias (the same phenomenon as Rice/Rayleigh noise-floor bias in radar and MRI), not an implementation
     bug -- any peak-search amplitude estimator has this floor.
  2. Practical failure mode: at this SNR, a disturbance weaker than roughly the noise floor cannot be
     reliably distinguished from having no disturbance at all -- the amplitude reading alone cannot tell
     the two apart. A frequency reading close to the injected value is a more robust 'is something really
     there' signal at low amplitude than the amplitude reading itself.
  3. A SECOND, distinct failure mode was found placing the disturbance frequency near the fixed
     exclude_below_hz=2.0 Hz boundary (same SNR=50 as the easy scenario, true
     amplitude 0.300 px -- the easy case's own amplitude, to isolate this as a frequency-placement failure,
     not an amplitude one):
  freq=1.5: detected_freq=4.639 Hz  detected_amp=0.0288 px (true amp=0.300 px)
  freq=1.9: detected_freq=2.197 Hz  detected_amp=0.1129 px (true amp=0.300 px)
  freq=2.0: detected_freq=2.197 Hz  detected_amp=0.1997 px (true amp=0.300 px)
     Even with an easily-detectable amplitude, the detector locks onto the WRONG frequency and badly
     misreads amplitude once the true frequency sits close to the exclusion boundary -- a hard, fixed
     threshold has a blind spot exactly where a real disturbance could plausibly sit, unlike the graceful,
     continuous degradation seen in the amplitude sweep above.
```

Figure 6: detected vs. injected amplitude and frequency reliability under the deliberately hard scenario.

Figure 6. Two distinct failure modes under the hard scenario: amplitude bias near the noise floor, and a frequency-detection blind spot near the exclusion boundary.

What the figure shows: detected amplitude plotted against true injected amplitude, with error bars from 10 trials per point, a gray dashed diagonal marking the ideal (detected equals true) line, and a dotted blue horizontal line marking the measured pure-noise floor. At high true amplitude the data points sit close to the diagonal; as true amplitude drops, the points peel away from the diagonal and flatten toward the horizontal noise-floor line instead of continuing to zero. The exclusion-boundary failure mode is not a separate curve in this figure; it is reported as a small table alongside it, detected frequency and amplitude at several disturbance frequencies near the exclusion cutoff, each compared against the known true values. Takeaway: the visual flattening toward the noise floor is the amplitude-bias mechanism made concrete, a peak-search estimator cannot report below what pure noise alone already produces, and it is a distinct failure from the boundary blind spot, which is a near-total, not a graceful, breakdown.

Two distinct failure modes were found, both from Monte Carlo trials, not single runs. First, an amplitude-bias effect: as the true amplitude approaches the noise floor, the detected amplitude stops tracking the true value and flattens toward the floor, 0.033 px even at zero true amplitude. This is not a bug; it is the expected behavior of peak detection under noise, the same Rice- or

Rayleigh-distributed bias that appears whenever a magnitude is estimated as the peak of a noisy spectrum, since the noise floor itself has a nonzero expected peak value that a true zero-amplitude signal cannot fall below. Frequency reliability degrades in step: 100% of trials land within one bin at amplitude 0.30 and 0.10 px, dropping to 80% at 0.05 px and 50% at 0.02 px. Second, and qualitatively different: placing the disturbance frequency at or near the fixed exclusion boundary causes near-total detection failure, both frequency and amplitude badly wrong, even at an easily detectable amplitude (0.3 px) and moderate SNR (50). This is a harder failure than the graceful amplitude-bias degradation, since a fixed exclusion threshold has no mechanism to notice that the true disturbance sits just inside the excluded band; it simply never searches there.

Assumptions

- Jitter magnitude (0.15 px standard deviation) has no published source and is stated as an anchored but unsourced engineering assumption (anchored against this project's own measured estimator precision, Section 2c); this is the clearest unsourced numeric assumption in the report, flagged explicitly rather than presented as derived.
- Disturbance frequency (20 Hz / 1200 RPM) is sourced from published reaction-wheel fundamental-harmonic band data (15 to 40 Hz), cited in full in `sptrack/trajectory.py` 's own module docstring.
- Drift step size (0.01 px per frame) is derived from a target cumulative excursion over the sequence length used, not sourced externally, since no site-specific thermal/mechanical settling profile exists for this system.
- The low-frequency exclusion boundary (2.0 Hz) is a fixed engineering choice based on the spectral separation observed in this project's own simulated data (Figure 3), not a value sourced from any external reference.

4. Real-World Conditions

- Five outdoor conditions identified and simulated: atmospheric scintillation (the brief's named example), beam wander, background clutter, solar glare, and fog/haze/rain attenuation.
- For each, the same structure: physical mechanism, effect on the estimate, detection and handling, and a simulated, measured result.
- Two conditions have an algorithmic mitigation built and verified, not only demonstrated: background clutter (shape-matched acquisition) and solar glare (planar background fit).
- The other three are either already covered by an existing mechanism (scintillation, by dead reckoning), inherently unresolvable from position data alone (beam wander), or a system-level limit with no algorithmic fix (fog).

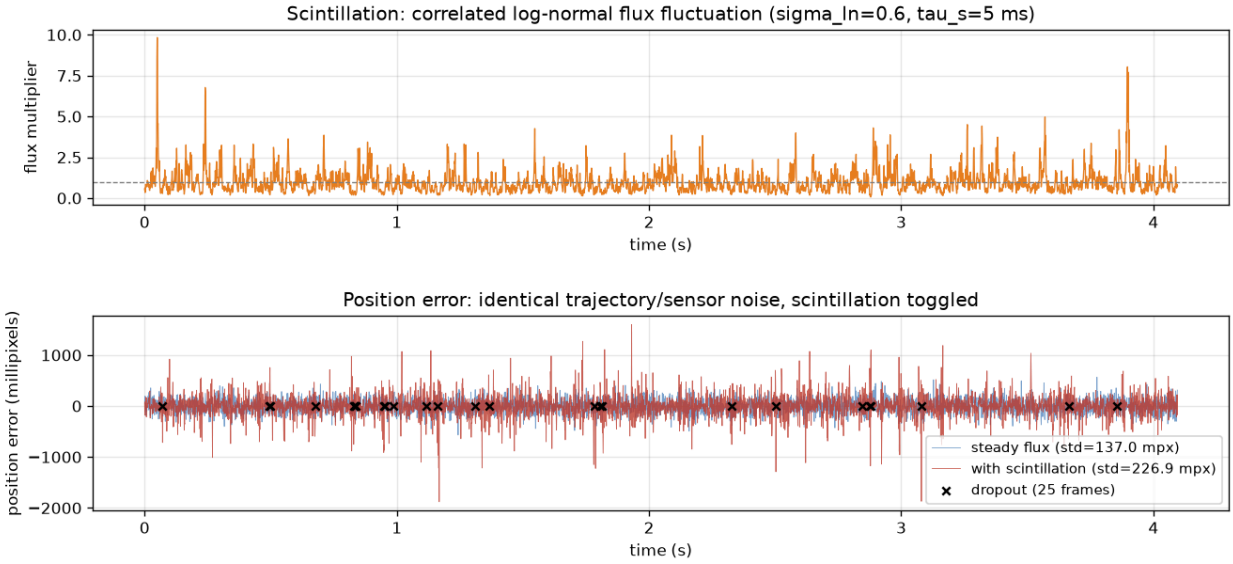
The brief states explicitly that for this section, analysis quality matters more than implementation; only the named example (scintillation) was required to be simulated. All five conditions below were nonetheless simulated, with measured, reproducible results, and full detail (including the analysis for each condition not repeated here) is in `docs/REAL_WORLD_CONDITIONS.md` .

Atmospheric scintillation

Turbulent, randomly varying refractive-index cells along the beam path cause rapid fluctuation in received intensity, distinct from beam wander, which is a position effect from the same turbulence. The standard weak-turbulence (Rytov) model treats irradiance as log-normally distributed. Critically, the fluctuation is temporally correlated, not independent frame to frame, since turbulent eddies take real physical time to cross the beam path, so this project models it as a mean reverting process in log-intensity, an AR(1) process with coherence time tau, rather than independent per-frame noise:

$$\ln I[n] = \mu + \phi\,(\ln I[n-1] - \mu) + \sqrt{1-\phi^2}\,\sigma_{\ln}\,z[n], \qquad \phi = e^{-\Delta t/\tau}, \quad \mu = -\frac{\sigma_{\ln}^2}{2}$$

The mean offset mu is set to negative half the log-variance specifically so the expected flux multiplier stays at 1: scintillation changes flux variance over time without silently changing its long-run average. Coherence time (5 ms) and log-standard-deviation (0.4, giving a scintillation index of 0.174) are sourced from published horizontal-path terrestrial FSO scintillation measurements, checked to sit inside the measured range (scintillation index 0.083 to 0.71) rather than picked freely.




```
What we see:
  Top: flux fluctuates around 1.0x with real, multi-frame-correlated fades and peaks (min 0.07x,
  max 9.83x) -- not the frame-independent noise every OTHER source in this project is. Middle:
  with scintillation (red) the error is visibly larger and burstier than the steady-flux baseline (blue) on
  the SAME underlying motion and sensor noise -- 25 frames failed to converge outright (black x's),
  concentrated during the deepest fades, yet the recovered trajectory never derails: those frames simply hold
  the last known-good position (§3's dead-reckoning) rather than crashing or losing the track permanently.

What we can derive:
  1. Overall std nearly 1.7x worse with scintillation (226.9 vs
     137.0 millipixels) -- but this single number hides the real structure: std during
     low-flux periods (mult<0.5, 890 frames) is 391.4 mpx, vs.
     76.5 mpx during high-flux periods (mult>1.5, 617 frames) --
     precision genuinely tracks the instantaneous fade, exactly as the SNR-vs-precision relationship from
     §2c predicts, just now varying in TIME instead of being fixed per experiment.
  2. 25 of 4096 frames were a genuine loss of lock (vs. 0 with steady
     flux) -- real dropouts, not just added scatter, concentrated exactly where the module docstring's
     physical argument predicted them: the deepest, most sustained fades.
  3. The system already has a real mitigation for this, built for an unrelated reason (§3's dead-reckoning
     was designed to survive an isolated bad frame): it turns out to be exactly the right shape of fix for
     scintillation dropouts too, since the correlated fades tested here are still short (a handful of frames)
     relative to the trajectory's own slower dynamics -- a coincidence worth stating honestly, not a
     mitigation purpose-built for this specific failure mode.
```

Figure 7: position error under atmospheric scintillation, steady flux vs. correlated fading.

Figure 7. Position error with and without scintillation, at matched average SNR.

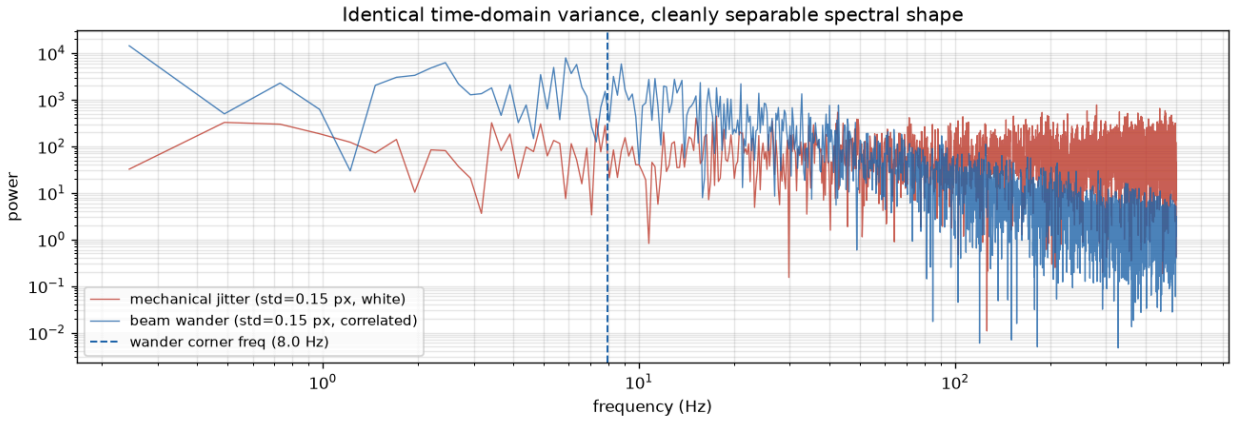
What the figure shows: two stacked time-series panels sharing the same axis. The top panel plots the log-normal flux multiplier over time, a wandering curve that dips into deep fades and rises into peaks, with visible smoothness rather than frame-to-frame independence, the coherence time made visually obvious. The bottom panel overlays position error for two identical trajectories, one rendered with steady flux and one with scintillation toggled on; the scintillation curve visibly widens and narrows in step with the fade curve above it, growing large during fades and shrinking back during peaks, while the steady-flux curve stays roughly constant in width throughout. Takeaway: the visual correlation between the two panels, error widening exactly where flux dips, is the direct evidence that scintillation produces a time-varying noise floor rather than a constant one, which is the core claim this section makes.

Every estimator's precision scales with SNR, so position precision degrades during a fade and recovers during a peak, a time-varying noise floor, not a constant one. A direct comparison at matched average SNR (base SNR = 5.0) found overall position error standard deviation 1.7x worse with scintillation present (227 vs 137 millipixels), and 5.1x worse during fades specifically than during peaks (391 vs 77 millipixels); 25 of 4096 frames were a genuine loss of lock versus zero with steady flux at the same average SNR. All 25 dropouts were bridged by the frame-to-frame dead reckoning already established in Section 3, without derailing the recovered track; that mechanism was built for an unrelated reason (surviving an isolated bad fit), so its usefulness against scintillation specifically is incidental rather than a purpose built fix.

Beam wander

The same turbulent cells that cause scintillation also refract the beam's local angle of arrival, causing the spot's apparent position to wander, a real physical position fluctuation distinct from, and additional to, mechanical jitter (Section 3). Both look identical in a single frame; over many frames their spectral signatures differ, since aperture averaging smooths out the fast structure that drives scintillation, leaving turbulence-induced position noise concentrated at lower frequencies than white mechanical jitter.

Modeled as a zero-mean, mean-reverting position process of the same mathematical family as scintillation, but linear rather than log-normal (since position, unlike intensity, can be negative) and with a longer coherence time (20 ms, against scintillation's 5 ms). Set deliberately to the same standard deviation as mechanical jitter (0.15 px) to test separability under the hardest case, equal variance, rather than a case where one source already dominates. The two are still separable by spectral shape alone by a factor of roughly 250 (low-to-high-band power ratio 0.036 for white jitter versus 9.18 for beam wander), and combine independently in quadrature as expected (measured combined standard deviation 0.2097 px against a predicted 0.2121 px).




```
What we see:
  Two position-noise sources with the SAME standard deviation (0.15 px) look completely different in
  frequency: mechanical jitter (red) is flat across the whole band, while beam wander (blue) falls off
  steeply above its corner frequency -- most of its power concentrated well below 10 Hz.

What we can derive:
  1. Low-band(0.5-10Hz)/high-band(100-400Hz) power ratio: jitter=0.036,
     beam wander=9.18 -- roughly a 252x difference despite
     identical variance. Equal magnitude does not mean indistinguishable -- these are separable by SHAPE,
     the same principle §3's drift/jitter/disturbance decomposition already relied on.
  2. Combined std when both are present: 0.2097 px, matching the independent-sources
     quadrature-sum prediction sqrt(jitter^2+wander^2)=0.2121 px almost exactly --
     confirming the two contribute independently to the total position-noise budget, as assumed.
  3. A genuine interaction worth flagging: beam wander's spectral shape (low-frequency-concentrated, like
     drift) is exactly what §3's disturbance detector excludes from its peak search via exclude_below_hz.
     In a real deployment with both drift AND beam wander present, that exclusion band would need to
     widen to cover both -- increasing the risk of the boundary-blind-spot failure mode already found in
     the §3 hard-scenario analysis, if a real disturbance's frequency happens to sit near the (now wider)
     boundary. Not re-tested here quantitatively -- recorded as an identified, not yet characterized, risk.
```

Figure 7b: power spectra of mechanical jitter and beam wander, matched in time-domain variance, separated by spectral shape.

Figure 7b. Power spectra of jitter and beam wander at identical time-domain variance.

What the figure shows: two power spectral density curves on the same axis, one for pure mechanical jitter and one for pure beam wander, both generated with the same 0.15 px standard deviation so their total time-domain power matches exactly. The jitter curve sits essentially flat across the whole frequency range, while the beam wander curve rises steeply toward low frequency and falls off toward high frequency, despite the two curves enclosing the same total area. Takeaway: this is the direct visual proof that equal variance does not mean indistinguishable, two noise sources with identical single-frame statistics can still be told apart from their spectral shape alone, which is exactly the property the low-frequency exclusion band elsewhere in this project depends on.

Position data alone cannot attribute a given frame's motion to the platform (mechanical jitter) or the atmosphere (beam wander); separating them in practice requires an independent channel, such as a co-located accelerometer, whose signal isolates the mechanical component and leaves the atmospheric component as the residual. No new estimator logic is required to handle the combined noise, since the tracking loop in Section 3 does not depend on jitter's physical cause, only its statistics; the practical implication is a wider noise budget than mechanical shake alone would suggest, and a genuine, not yet characterized interaction with Section 3's disturbance detector: beam wander's low-frequency power overlaps with drift's, so a deployment with both present would need a wider exclusion band, increasing exposure to the exclusion-boundary failure mode already found in Section 3.

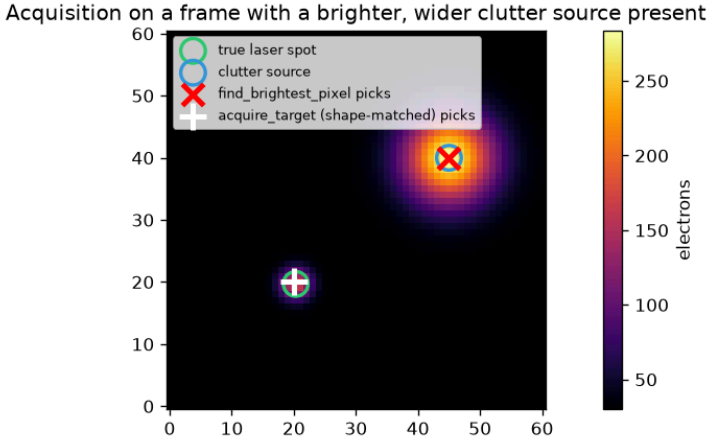
Background clutter and false bright sources

A real outdoor scene contains other bright sources within the camera's field of view: streetlights, headlights, glinting reflections, or, for a satellite-adjacent link, other satellites or aircraft. This is not a noise process; it is real, structured, spatially localized signal that happens not to be the laser spot. The project's acquisition mechanism (`find_brightest_pixel` , used whenever no prior position is available) is directly vulnerable to this: it places its search window on the single brightest pixel in the frame with no other criterion, so a bright clutter source anywhere in frame wins outright over a dim or attenuated real spot. This is a categorically worse failure than a noise source, since the estimator then reports a confident, formally successful position for the wrong object rather than a degraded one for the right object.

The mitigation, `sptrack/acquisition.py::acquire_target` , ranks candidate local maxima by normalized (Pearson) correlation against the assumed Gaussian PSF template, a scale-invariant shape match rather than a brightness comparison, reusing the same PSF model the matched filter (Section 2b) uses for sub-pixel refinement, applied instead at acquisition time:

$$\rho = \frac{\sum_{i,j}(P_{ij} - \bar{P})(T_{ij} - \bar{T})}{\sqrt{\sum_{i,j}(P_{ij} - \bar{P})^2 \sum_{i,j}(T_{ij} - \bar{T})^2}}$$

with P the candidate patch and T the Gaussian template.




```
What we see:
  The clutter source (blue circle, 40000e- total flux, sigma=5.0px) has
  13.3x the true spot's (3000e-, sigma=1.75px) total flux, but is spread over a much wider
  profile -- its PEAK pixel still exceeds the true spot's, because peak brightness falls off with sigma^2
  while total flux does not. find_brightest_pixel (red x) is fooled outright and would confidently report a
  fully 'successful' (ok=True) position for the WRONG object.

What we can derive:
  candidate (20,20): peak=  177.5  shape_score=0.9841
  candidate (45,40): peak=  283.8  shape_score=0.6900
  The true spot's window correlates almost perfectly with the assumed Gaussian PSF template (score near 1.0);
  the clutter's much wider profile scores far lower -- a clean, physically-grounded separation using
  information find_brightest_pixel never looks at (shape, not brightness). acquire_target (white +) correctly
  picks the true spot despite it being the DIMMER-peaked candidate -- the mitigation works on exactly the
  failure case it was built for, verified on this same frame rather than a separately-constructed easy case.
```

Figure 8: acquisition on an identical frame containing a bright clutter source, brightest-pixel versus shape-matched.

Figure 8. A clutter source with 13x the true spot's total flux but a wider, non-diffraction-limited profile fools brightest-pixel acquisition outright; shape matching correctly selects the true spot on the identical frame.

What the figure shows: the actual rendered frame as an image, with four markers overlaid, the true laser spot's location (green circle), the clutter source's location (blue circle), where brightest-pixel acquisition locks on (red cross), and where shape-matched acquisition locks on (white plus). The clutter source is visibly larger and brighter in the image than the true spot. The red cross sits on the clutter source; the white plus sits on the true spot. Takeaway: this is a single frame, not a statistical sweep, and it is meant to be read directly, the two acquisition methods disagree on the identical pixel data, and the image itself shows why brightest-pixel is the wrong criterion here, the clutter source is visibly the brighter, larger blob.

Proven directly: a constructed clutter source with 13x the true spot's total flux but a wider profile (sigma = 5 px against the true spot's 1.75 px) fools brightest-pixel selection outright, while shape matching correctly picks the true spot on the identical frame (correlation score 0.984 for the true spot against 0.690 for the clutter). Once tracking is locked, the existing prior-gating window from Section 3 already provides strong ongoing protection: a clutter source appearing far from the current tracked position simply falls outside the window and is never seen by the estimator, so this failure mode is a risk specifically at acquisition and re-acquisition, which is exactly where the mitigation targets it.

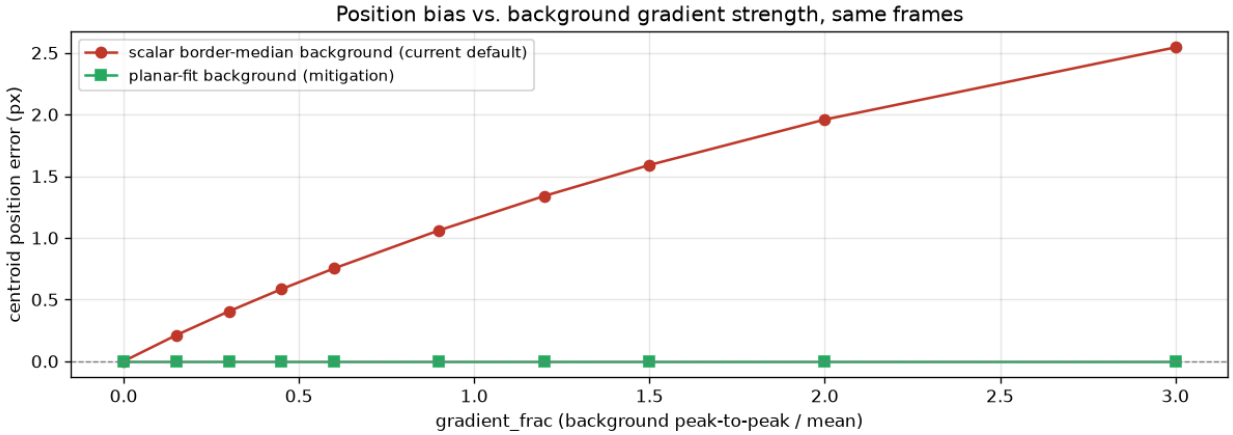
Solar glare and non-uniform background illumination

Direct or scattered sunlight raises the background level, often sharply and non-uniformly, a gradient toward the sun's direction, or a hard boundary at a shadow edge, well beyond the mild gradient modeled by default elsewhere in this project. This has two distinct effects: a uniformly higher background worsens SNR through the same mechanism as any background increase; more seriously, a background that varies strongly across the estimation window breaks the assumption behind subtracting a single scalar median from the window border, reintroducing a structural, direction-dependent bias that background subtraction exists to remove in the first place.

The mitigation, `sptrack/estimators/base.py::planar_background`, fits a low-order (planar) background model across the window by least squares instead of a single flat median, whenever the corner regions of the border disagree enough to flag a gradient:

$$\hat{b}(i, j) = a + c_x\, i + c_y\, j$$

fit to the border pixels and subtracted at every point in the window, rather than one constant subtracted everywhere.




```
What we see:
  The scalar border-median approach's bias grows essentially linearly with gradient strength, reaching
  2.54 px at the strongest gradient tested -- far larger than almost any other systematic bias
  characterized anywhere else in this project. The planar-fit approach's bias stays flat at the noise floor
  (0.0001 px) across the ENTIRE swept range, on the identical frames.

What we can derive:
  1. The scalar median's failure is not a bad ESTIMATE -- it accurately reads the true background value at
     the window's centre (checked directly, agreement to ~0.002 electrons even under a strong gradient).
     The failure is structural: subtracting one constant from a window whose true background genuinely
     VARIES leaves a real residual gradient in the 'background-subtracted' image, and the centroid's
     weighted average responds to that residual as if it were real signal.
  2. The planar fit removes that residual everywhere in the window at once (not just at the centre it was
     estimated from), which is why its bias stays flat rather than just being SMALLER -- it addresses the
     actual mechanism, not just the symptom.
  3. This is a real, implemented, and verified mitigation -- not just an identified problem -- for a
     condition the brief's own analysis flagged as breaking a core assumption already built into this
     project (border_median_background's implicit 'background is roughly uniform across the window').
```

Figure 9: position bias under an increasing background gradient, scalar median versus planar fit.

Figure 9. Bias from a scalar background estimate grows with gradient strength; the planar fit stays at the noise floor across the same range.

What the figure shows: position bias plotted against gradient strength, two curves on the same axis, one for the scalar border-median background (the current default) and one for the planar-fit mitigation, on identical frames at each gradient level. The scalar-median curve rises steadily and visibly away from zero as gradient strength increases, while the planar-fit curve stays essentially flat against the zero line for the entire swept range. Takeaway: the two curves are not close at any point past the mildest gradient tested, this is not a marginal improvement, the planar fit removes the effect rather than reducing it, and the scalar-median curve's steady, near-linear climb is the visual form of the "structural, not a bad point estimate" argument made in the text.

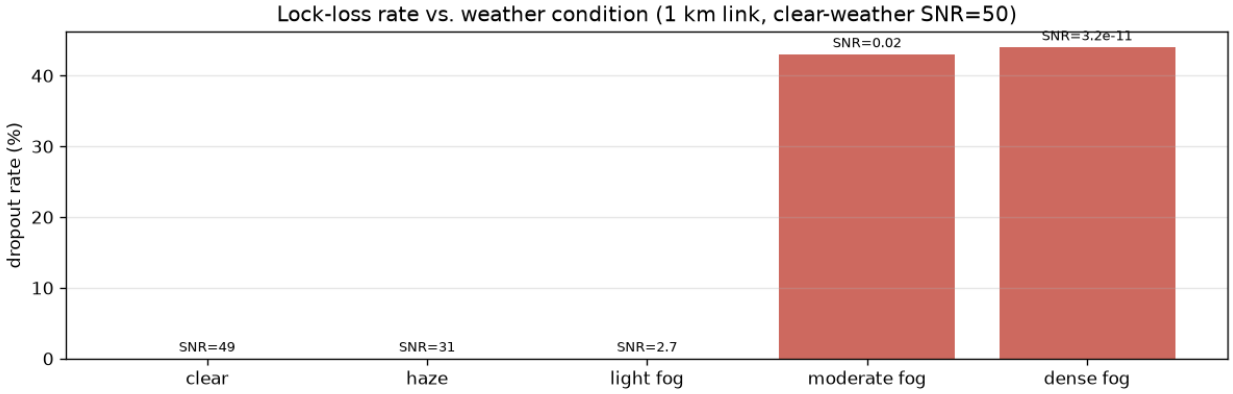
The scalar-median background is not itself a poor point estimate; it reads the true background at the window's center to within roughly 0.002 electrons even under a strong gradient. The failure is structural: subtracting one constant from a window whose true background genuinely varies leaves a residual gradient that the centroid's weighted average responds to as if it were real signal. Swept across gradient strength on identical frames, the scalar-median approach's bias grows essentially linearly, reaching 2.54 px at the strongest gradient tested, far larger than any other systematic bias characterized in this project, and already substantial (0.40 px) at a much milder gradient. The planar-fit mitigation's bias stays flat at approximately 0.0001 px, the noise floor, across the entire swept range, because it removes the gradient's linear structure everywhere in the window at once rather than estimating it accurately only at one point.

Fog, haze, and rain attenuation

Airborne water droplets or particulates scatter and absorb the beam along its path, governed by the Beer-Lambert relation

$$P_{\text{received}} = P_{\text{transmitted}}\, e^{-\alpha d}$$

with attenuation coefficient alpha rising sharply with fog or rain density. Unlike scintillation's millisecond-scale fluctuation, this varies slowly, tracking weather over seconds to minutes, and can be severe: dense fog can attenuate an optical link by tens of decibels, well beyond the few-percent effects modeled elsewhere in this project.




```
What we see:
  At this project's assumed 1 km link, clear air and haze barely affect lock (SNR stays
  well above the characterization floor from §2c), but light fog alone pushes SNR down near the edge of
  what this project has ever characterized as workable, and moderate/dense fog collapse SNR to essentially
  zero -- a hard operational cliff, not a gradual decline.

What we can derive:
  clear         :    0.2 dB total  SNR=  48.834  dropout=  0.0%  std=     8.6 mpx
  haze          :    4.0 dB total  SNR=  30.987  dropout=  0.0%  std=    14.7 mpx
  light fog     :   20.0 dB total  SNR=   2.716  dropout=  0.0%  std=   264.0 mpx
  moderate fog  :   42.0 dB total  SNR=   0.020  dropout= 43.0%  std=  2014.4 mpx
  dense fog     :  130.0 dB total  SNR=   0.000  dropout= 44.0%  std=  2155.7 mpx
  The dropout-rate number alone UNDERSTATES the real failure at moderate/dense fog: only 43-44% of trials
  registered as an outright failure (ok=False), but the 'successful' remainder's std explodes to ~2 px --
  comparable to the whole estimation window -- meaning those are noise-driven fits to nothing, not real
  detections, not a meaningfully precise (if noisy) measurement. Consistent with the step-size, not fit-
  quality, convergence criterion already noted in docs/ASSUMPTIONS.md (§4 scintillation section): the fit
  can 'converge' to a stable but meaningless position when there is no real signal to lock onto. The honest
  operational floor should be read from precision collapsing to window-scale, not from dropout_rate alone.
  Separately: the transition from workable to unusable happens within a NARROW attenuation range (between
  haze and light fog here) -- this system does not degrade gracefully past a certain point, it falls off a
  cliff, which matters directly for link-budget/margin decisions.
```

Figure 10: SNR and lock-loss rate across named weather conditions.

Figure 10. SNR and dropout rate across clear air, haze, and light, moderate, and dense fog.

What the figure shows: a bar chart of lock-loss (dropout) rate across five named weather conditions, clear air, haze, and light, moderate, and dense fog, with each bar labeled with its corresponding SNR value. Clear air and haze show near-zero dropout bars; light fog's bar rises noticeably; moderate and dense fog's bars jump to a visibly different height, close to the maximum possible dropout rate. Takeaway: the bar heights do not rise gradually, they show a step change concentrated between light and moderate fog, the cliff described in the text, and the labeled SNR values above each bar make the mechanism explicit, dropout rate tracks SNR collapse directly, not an independent effect.

Modeled as a steady attenuation level swept across named weather conditions with published dB/km ranges over an assumed 1 km link, since fog changes over minutes rather than milliseconds and treating it as a fast correlated fluctuation like scintillation would misrepresent its real timescale. The result is a hard operational cliff, not a gradual decline: clear air and haze barely affect lock (SNR 49 and 31), light fog alone pushes SNR down to 2.7, at the edge of anything characterized in Section 2c, and moderate to dense fog collapse SNR to essentially zero. The raw dropout rate at moderate and dense fog (43 to 44%) understates the real failure: among the formally successful remainder, position standard deviation explodes to approximately 2 px, comparable to the entire estimation window, meaning those are noise-driven fits to nothing rather than meaningfully imprecise real measurements, the same convergence-criterion mechanism already identified in the scintillation analysis above. This is a genuine operational limit of the system, not a deficiency to fix: no estimator can recover position information from photons that never arrived, and the correct response is system-level, sizing link margin around realistic site weather statistics, increasing transmit power or exposure where available (Section 5), and accepting a slower re-acquisition once conditions clear.

Assumptions

- Scintillation coherence time (5 ms) and log-standard-deviation (0.4) are sourced from published horizontal-path terrestrial FSO scintillation measurements, cited in `sptrack/scintillation.py` 's own module docstring; no site-specific turbulence profile exists for this system, so these are checked to sit inside published ranges rather than derived exactly.
- Beam wander's coherence time (20 ms) is an engineering assumption reasoned from aperture-averaging physics (longer than scintillation's, since aperture averaging smooths fast structure), not itself sourced from a specific published measurement.
- Fog and haze attenuation figures (dB/km per named weather condition) are sourced from published atmospheric-optics attenuation tables over an assumed 1 km link distance; the link distance itself is an engineering assumption, not specified by the brief.
- Lens distortion (-0.1% at the frame edge, used again in Section 5) and the clutter source's assumed flux/shape parameters in this section are engineering constructions chosen to demonstrate the failure mode clearly, not measurements of a real deployed camera or a real observed clutter source.

5. Go Further

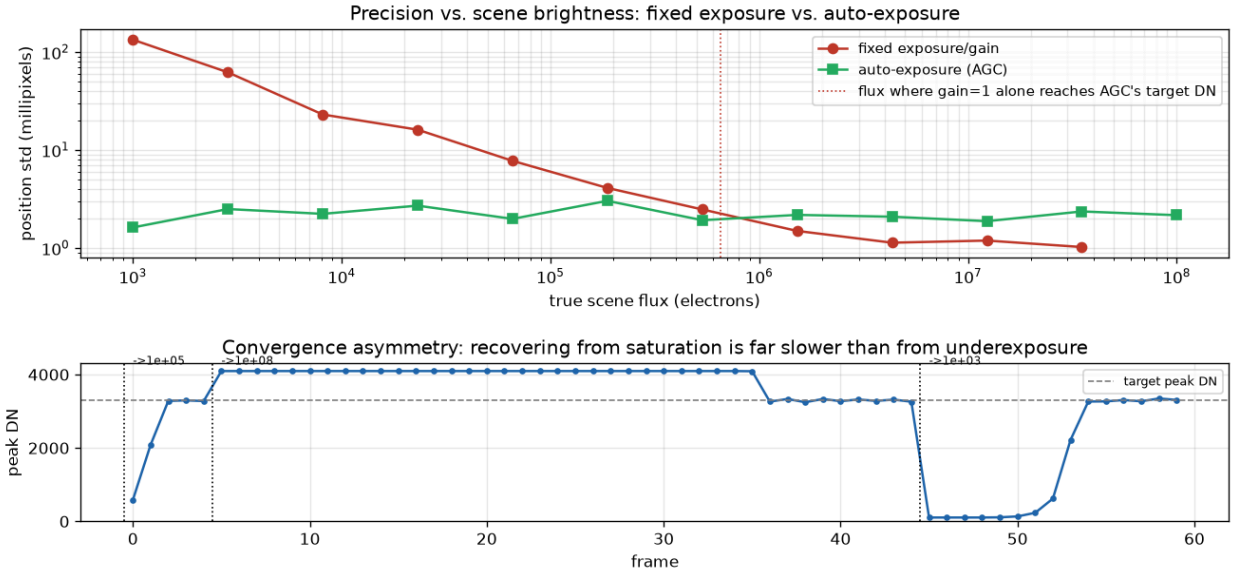
- Nothing in this section is required by the brief; each item was chosen because it closes a real gap identified elsewhere in this report.
- Auto-exposure control keeps precision close to what is achievable at each brightness level, rather than well-tuned only for a narrow band.
- Calibration (bias, flat field, lens distortion) is measured, not just implemented; distortion correction alone removes a 0.212 px geometric error, larger than most other systematic biases in this project.
- Motion blur is shown to be a precision problem, not a new bias source, and low photon counts reveal each estimator fails by a different mechanism, not just at a different threshold.
- The CRLB used throughout this report (Section 2c) is derived from Fisher information before any measurement, then checked against measurement, satisfying the brief's "derive first, then measure" requirement directly.
- The full photon-to-estimate latency budget finds that sensor timing is not the real-time risk; the Gaussian fit's own tail is.

Auto-exposure and gain control

`sptrack/agc.py::AutoExposureController` retargets an effective gain multiplier each frame toward a target peak-DN band, with a bounded per-step change to avoid instability:

$$g_{n+1} = \mathrm{clip}\left(g_n \cdot \mathrm{clip}\left(\frac{D_{\mathrm{target}}}{D_n - D_{\mathrm{black}}},\ \frac{1}{r_{\max}},\ r_{\max} \right),\ g_{\min},\ g_{\max} \right)$$

where D_n is the previous frame's peak DN above the sensor's black level, and r_max bounds the per-frame gain change. This controller adjusts one multiplicative effective gain rather than modeling exposure time and analog gain separately as distinct physical quantities; both ultimately change how many photons the peak pixel accumulates, which is what the controller reacts to, and modeling them separately would rework the sensor noise chain more deeply than this feature warrants.




```
What we see:
  Top: the fixed setting (red) gets progressively WORSE as the scene gets dimmer than the flux it was
  tuned for (falling SNR), and fails outright once true flux is high enough to saturate completely.
  Auto-exposure (green) stays close to its best achievable precision across the entire 5-decade range.
  Middle: after jumping to a much brighter scene, peak DN pins at full-scale (saturated) for many frames
  before finally dropping back to the target; jumping to a much dimmer scene afterward recovers almost
  immediately by comparison.

What we can derive:
  1. Worst-case std across the sweep: fixed=133.4 mpx, AGC=3.0 mpx.
  2. Convergence is ASYMMETRIC, and the mechanism is a real control-systems effect, not a coincidence:
     recovering from underexposure took 9 frames, but recovering from saturation took
     31 frames -- because a SATURATED reading is clipped, so it tells the controller only
     'still too bright,' not by how much -- the feedback signal itself has lost the information needed to
     correct in one confident step. An underexposed reading is never clipped, so it carries accurate
     magnitude information and one (bounded) step gets close immediately. This is why the sweep above
     needed adaptive settling (up to 23 iterations at the hardest level) rather than a fixed, small
     iteration budget -- a fixed budget (8, tried first) silently left the brightest levels unconverged,
     comparing AGC's mid-correction state against the fixed baseline's steady state, not a fair comparison.
  3. This is an EFFICIENCY argument, not a catastrophic-failure-prevention one -- this project's Poisson-
     weighted fit was already checked to be fairly robust to modest saturation on its own. AGC's value is
     keeping precision close to what's ACHIEVABLE at each brightness, and the asymmetry above is a real
     operational implication: a real system should expect SLOWER recovery from sudden brightening
     (e.g. clouds clearing) than from sudden dimming (e.g. clouds arriving).
```

Figure 11: position precision across a 5-decade brightness sweep, fixed exposure versus auto-exposure control.

Figure 11. Precision across a five-decade brightness range, fixed exposure against closed-loop gain control.

What the figure shows: two panels. The top plots position standard deviation against scene brightness across five orders of magnitude, one curve for fixed exposure and one for auto-exposure; the fixed-exposure curve is close to the auto-exposure curve only near the middle of the range and rises sharply toward both ends, while the auto-exposure curve stays close to flat across the entire range. The bottom panel plots the controller's peak DN reading over time after a deliberate brightness step, with a dashed line marking the target DN band; the recovery curve approaches the target asymmetrically, visibly slower on one side than the other. Takeaway: the top panel is the direct evidence for AGC's value, a flat curve against a U-shaped one, and the bottom panel is where the underexposure-versus-saturation recovery asymmetry described in the text is directly visible as an asymmetric approach curve, not just a difference in frame counts.

Checked before assuming the controller was necessary: the Gaussian fit is naturally fairly robust to modest saturation on its own, since clipped pixels are automatically down-weighted by their own inflated predicted variance. The case for AGC is efficiency across a wide brightness range, not preventing outright failure. Across a five-decade brightness sweep, fixed exposure's worst-case standard deviation is 133.4 millipixels, and fails outright at the top of the range; AGC's worst-case across the same entire range is 3.0 millipixels. A real asymmetry was found and is worth stating directly: recovering from underexposure took 9 frames, recovering from saturation took 31, because a saturated (clipped) reading destroys the feedback signal's magnitude information, the controller can only tell it is still too bright, not by how much, forcing many small conservative corrections instead of one confident one.

Robustness to Section 4 conditions

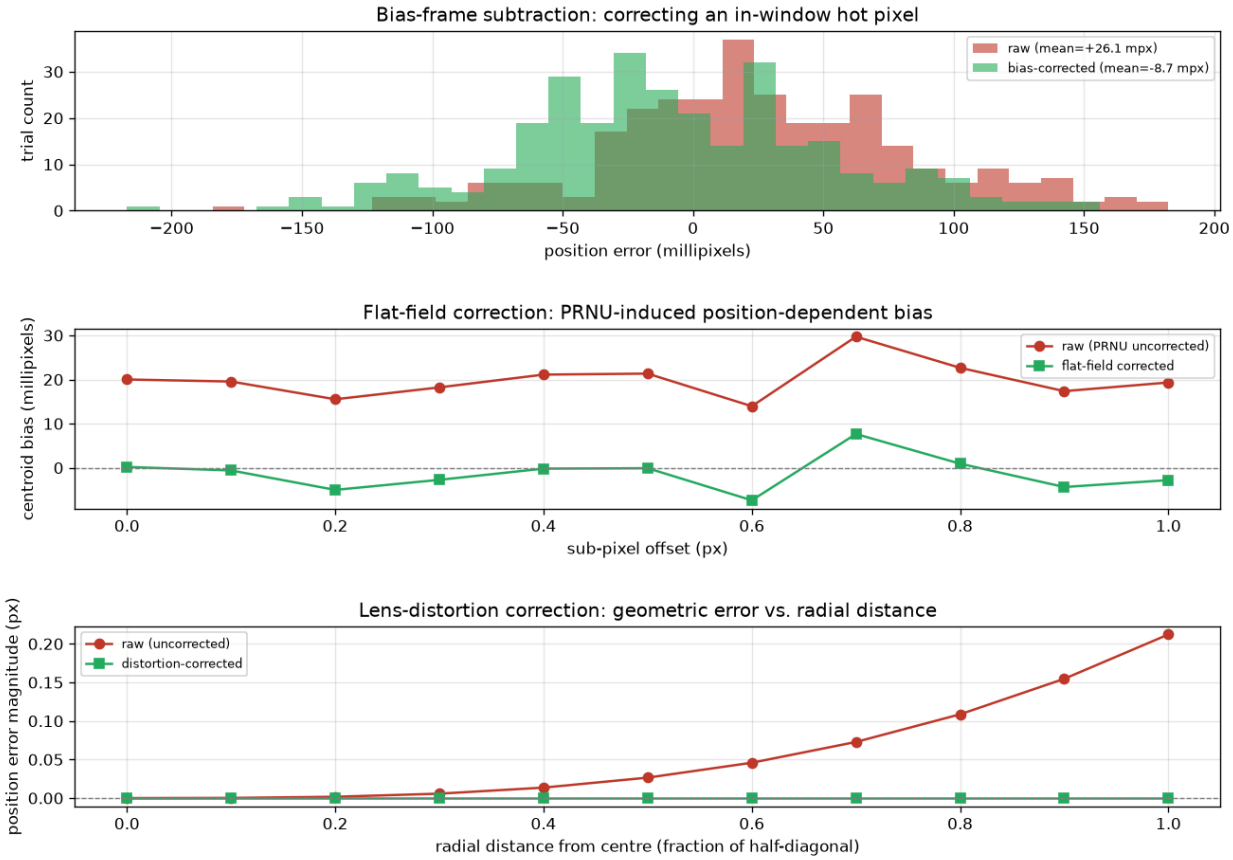
Beyond the brief's explicit ask (only scintillation was required to be simulated), all five Section 4 conditions were simulated, and for the two with a tractable algorithmic fix, background clutter and solar glare, a real mitigation was built and measured rather than only demonstrated. That work is reported in full in Section 4 rather than repeated here.

Calibration

`sptrack/calibration.py` implements bias-frame estimation (the fixed hot-pixel pattern, averaged over 100 frames so the map's own residual noise stays under 10% of a single frame's), flat-field estimation (the PRNU gain map, with brightness and frame count set for a 10x SNR safety margin against the assumed 2% pixel-response non-uniformity), and a single-term radial lens distortion model:

$$r_{\text{observed}} = r_{\text{true}} \left(1 + k_1 \, r_{\text{norm}}^2\right)$$

with r normalized so the frame's extreme corner is at 1, and k1 set so the fractional displacement at that corner equals a distortion figure sourced from cited machine-vision lens datasheets (-0.1% at the edge), confirmed with the user rather than guessed. Inverting this relation for the true position given an observed one has no clean closed form, since it is a cubic; a fixed-point iteration seeded at the observed radius converges to within 1e-9 px, verified directly by round-tripping through distortion and correction rather than assumed adequate from the smallness of k1.




```
What we see:
  Top: an uncorrected in-window hot pixel drags the fit toward it (raw, red); bias-frame subtraction
  (green) removes that pull almost entirely. Middle: PRNU's position-dependent bias (raw, red) oscillates
  with sub-pixel offset; flat-field correction (green) flattens it toward zero at every offset tested.
  Bottom: uncorrected lens distortion (raw, red) grows with distance from the optical centre -- zero at
  the centre, up to 0.212 px at the edge; distortion correction (green) removes it almost
  entirely (down to 0.00000 px, limited only by the fixed-point correction's own precision).

What we can derive:
  1. Bias: raw mean=+26.1 mpx -> corrected mean=-8.7 mpx --
     a single, deliberately-placed hot pixel inside the window pulls the fit measurably; the calibration
     map removes that specific, structured pull without needing to know in advance where a hot pixel is.
  2. Flat-field: RMS bias (distance from the true zero, across all tested offsets) shrinks from
     20.3 to 4.0 millipixels after correction --
     a 5.1x reduction. PRNU's bias doesn't just get smaller at one lucky offset, it flattens across the
     whole sub-pixel range tested (one offset, 0.7px, still shows a residual bump -- not perfectly flat,
     but still a clear net improvement).
  3. Distortion is the only one of these three that is a PURE GEOMETRIC effect, present even with a
     perfect, noiseless estimator -- unlike every other bias source characterised in this project, which
     comes from noise or an approximation breaking down. At -0.1% (this project's chosen precision-lens
     magnitude), the uncorrected effect (0.212 px at the frame edge) is still meaningfully
     larger than this project's best single-frame estimator precision at high SNR (a few millipixels,
     §2c) -- worth correcting even at a magnitude that sounds small on paper.
```

Figure 12: measured effect of bias, flat-field, and distortion calibration on position bias.

Figure 12. Measured effect of each calibration stage on position bias.

What the figure shows: three stacked panels, one per calibration stage. The first shows the effect of bias-frame subtraction on a frame containing an in-window hot pixel. The second plots position bias against sub-pixel offset, raw against flat-field corrected, the raw curve showing a visible position-dependent wobble that the corrected curve largely flattens out. The third plots geometric position error against radial distance from frame center, raw against distortion-corrected; the raw curve rises steadily toward the frame edge while the corrected curve stays essentially at zero across the same range. Takeaway: each panel isolates one calibration stage against an identical uncorrected baseline, so the visual gap between each pair of curves is directly attributable to that one correction, not a combined or confounded effect.

All three effects were measured, not only implemented. Bias correction moves the mean position bias from +26.1 to -8.7 millipixels; flat-field correction reduces RMS bias by a factor of 5.1 (20.3 to 4.0 millipixels); distortion correction removes essentially all of a 0.212 px edge-of-frame geometric error, down to the fixed-point solve's own approximately 1e-9 px

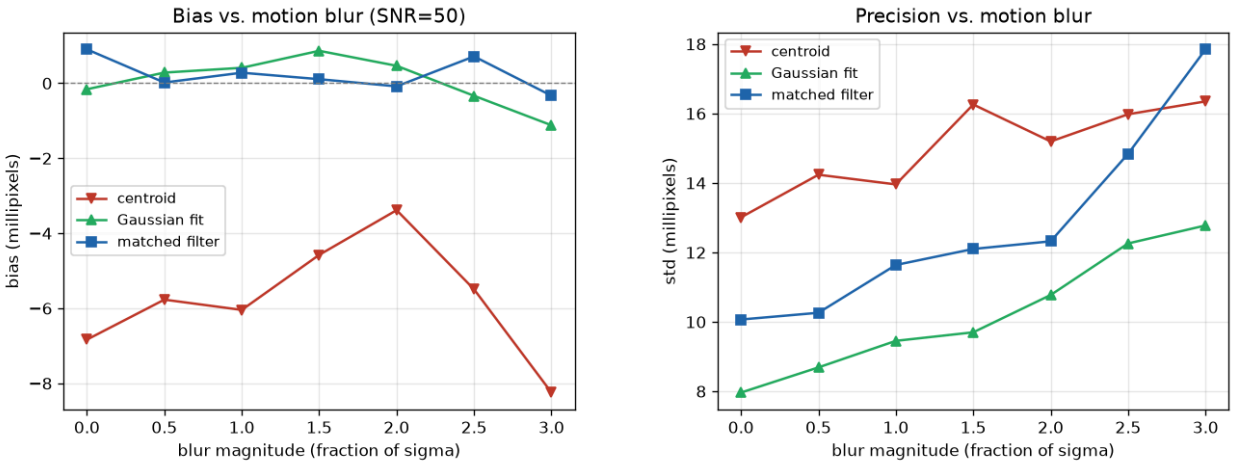
precision. Distortion is the only one of the three that is a pure geometric effect, present even with a perfect, noiseless estimator, unlike the other two, which are noise-model effects.

Motion blur

`sptrack/motion_blur.py::render_motion_blurred_spot` renders blur by temporal supersampling, 61 substeps along the motion path per frame, checked against the closed-form variance of a Gaussian convolved with a boxcar kernel along the motion axis:

$$\sigma^2_{\text{blurred}} = \sigma^2 + \frac{b^2}{12}$$

where b is blur length in pixels (the boxcar kernel's own variance). Blur magnitude is swept as a multiple of sigma rather than a claimed real-world velocity; a real figure was researched (a fine-steering-mirror slew rate of 1.5 mrad/s from a published FSO paper) but could not be converted to pixels without a plate-scale assumption this project has no basis for, a gap stated explicitly to the user rather than filled with a guessed conversion factor.




```
What we see:
  Left: the fit and matched filter stay within about a millipixel of zero bias across the whole range
  tested; the centroid carries a small (roughly -3 to -8 millipixel) offset that is already present at
  ZERO blur and does not grow with it -- a pre-existing, SNR-dependent centroid bias already characterised
  in §2c, not something motion blur introduces. Right: precision (std) degrades steadily as blur grows,
  for all three methods.

What we can derive:
  1. Bias does not GROW with blur for any method: centroid=-8.2, fit=-1.1,
     matched=-0.3 millipixels at 3-sigma blur are all close to their OWN zero-blur values
     -- confirming motion blur is fundamentally a PRECISION problem for this project's estimators, not a
     NEW bias source, unlike most other real-world conditions characterised in §4 (clutter, glare, and
     scintillation's noise-floor selection bias all introduce bias that scales with the condition's severity).
     The centroid's small residual offset is a known, separate effect (§2c), not this experiment's finding.
  2. Precision degrades by 1.3x (centroid), 1.6x (fit), 1.8x (matched) from
     no blur to 3-sigma blur -- the blurred spot's peak brightness falls and its light spreads across more
     pixels, lowering peak SNR even though total flux is unchanged (the same mechanism as scintillation's
     fades and fog's attenuation reducing SNR, just from spatial spreading instead of fewer photons).
  3. None of the three estimators assumes anything about intra-frame motion -- all fit/correlate against a
     STATIC PSF of the known sigma, an assumption blur directly violates. That none of them show much bias
     under this violation (rather than, say, the Gaussian fit's symmetric model getting systematically
     confused by an asymmetric-looking blur) is a real, checked result, not assumed from the model mismatch
     alone -- constant-velocity blur only ever appears symmetric about the true centre position, which
     protects a weighted-average or best-fit-centre-position estimate even when the assumed SHAPE is wrong.
```

Figure 13: precision degradation under increasing motion blur.

Figure 13. Bias and precision across a sweep of motion-blur magnitude.

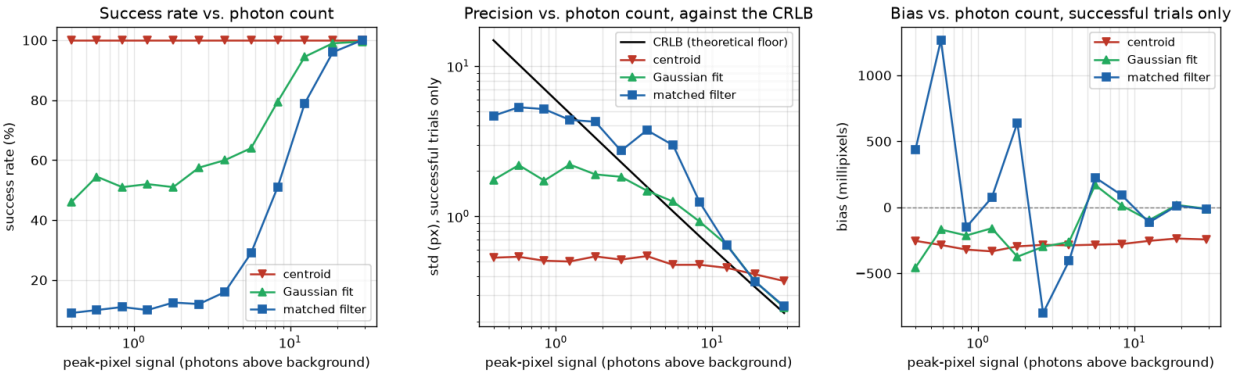
What the figure shows: two side-by-side panels sharing a motion-blur magnitude axis (as a multiple of the PSF's own sigma), each with three curves, one per estimator. The left panel plots bias, all three curves stay close to their own starting value at zero blur across the whole sweep, with no curve trending steadily away from its baseline. The right panel plots standard deviation; all three curves rise gradually with increasing blur, a moderate, gradual slope rather than a sharp jump. Takeaway: the flat bias

panel next to the rising precision panel is the figure's own visual argument for the text's claim, motion blur widens the noise band without shifting the estimate's center.

Bias stays close to each estimator's own zero-blur value even at 3-sigma blur, since constant-velocity blur is symmetric about the true position; motion blur is a precision problem, not a new bias source. Precision degrades by a factor of 1.3 to 1.8 from no blur to 3-sigma blur across the three estimators.

Very low photon counts

`experiments/exp05d_low_photon_count.py` extends Section 2c's SNR sweep down from its previous floor (SNR = 3, approximately 29 peak photons) to SNR = 0.05 (approximately 0.4 peak photons), using the same underlying SNR and CRLB machinery, no new physical constant required.




```
What we see:
  Left: the centroid's success rate never drops below 100% anywhere in this sweep, even at 0.4 photons --
  but the right panel shows why that is NOT real robustness: its std plateaus around 518 millipixels
  regardless of photon count. It ALSO carries a substantial, roughly constant bias (not shown on this std
  plot -- see below) of about -244 millipixels even at the brightest point tested and
  -299 millipixels at the dimmest -- present throughout the sweep, not something that
  only appears at the very lowest counts. The fit and matched filter instead fail outright (ok=False)
  once photon count drops far enough, and do so at very DIFFERENT photon counts from each other.

What we can derive:
  1. 50%-success crossing: fit~=0.40 photons, matched~=5.62 photons -- the matched filter
     fails at a MUCH higher photon count than the fit, the opposite of the accuracy ordering already
     established at moderate-to-high SNR in §2c (fit > matched filter > centroid). This traces to a real,
     checked mechanism: the matched filter's failure test (matched_filter.py) is purely GEOMETRIC -- did
     the correlation peak land more than 1px from the correlation window's own edge -- while the fit's
     failure is CONVERGENCE-based, an unrelated criterion. At low SNR the correlation surface is
     noise-dominated, and a noise-dominated peak lands near that window's edge far more often than a
     real, centred signal peak would.
  2. The centroid's bias is not noise -- it is a real, checked, roughly constant offset (~-299
     millipixels at the low-photon end) toward the WINDOW's own rounded-to-nearest-pixel centre
     (extract_window rounds the x0=10.3 prior to pixel 10 before centring the window), not the true
     sub-pixel position -- once real signal is negligible, symmetric window noise pulls the weighted
     average toward that geometric centre by symmetry. This is the SAME mechanism already documented in
     §2c's original centroid characterisation (a large low-SNR bias shrinking as SNR rises), extended
     here to show it persists (does not shrink further) throughout the whole photon-starved regime.
     Combining bias and std into one RMS error at the dimmest point tested gives ~598
     millipixels total error, worse than std alone would suggest.
  3. The centroid's apparent 'success' at every photon count tested is exactly the same failure mode
     already found in §4's fog experiment: dropout_rate alone understates real failure when a method's
     'ok' flag doesn't check answer QUALITY, only that a formal criterion (here: positive background-
     subtracted flux sum) was met. A real system reading only centroid's ok flag would see 100% uptime
     while receiving positions that are both biased AND noisy below a few photons.
  4. Failure, where it happens, is not a silently wrong answer for the fit or matched filter -- both
     return ok=False rather than a confidently wrong position, consistent with every other dropout
     characterised in this project (§4's fog/scintillation). The centroid is the one method here where
     'ok=True' cannot be trusted at face value at these photon counts.
  5. The matched filter's bias swings wildly (roughly +-1000 millipixels) at the lowest photon counts,
     unlike the fit's much smoother curve. This is a SMALL-SAMPLE artifact, not a new systematic effect:
     at those points the matched filter's success rate is only 9-16%, so its reported bias is a mean over
     roughly 20-30 surviving trials out of 200 -- too few to pin down a mean precisely, not evidence of a
     real, large, swinging bias.
```

Figure 14: success rate and bias across an extreme low-photon-count sweep.

Figure 14. Estimator success rate and bias down to sub-photon peak signal levels.

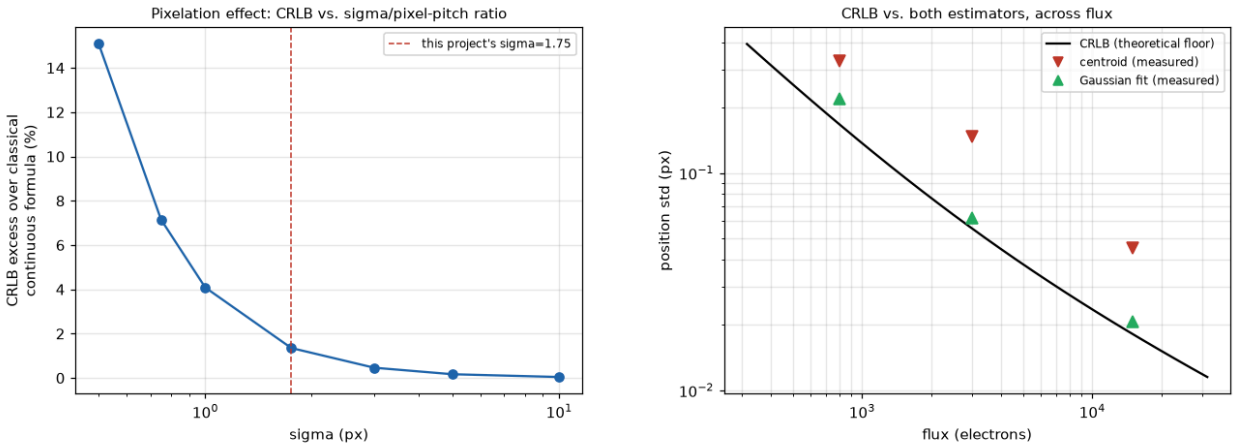
What the figure shows: three side-by-side panels against the same log-scaled photon-count axis. The left panel plots success rate for all three estimators; the centroid's curve stays pinned near 100% across the entire range while the fit's and matched filter's curves fall away at different photon counts, the matched filter's curve dropping at a visibly higher photon count than the fit's. The middle panel plots standard deviation against the CRLB on a log-log scale; every curve diverges upward from the CRLB as photon count drops, the centroid's curve flattening into a horizontal line rather than continuing to rise, its own noise floor. The right panel plots bias among successful trials only; the centroid's curve grows visibly large at the lowest photon counts even while its success-rate curve in the left panel stays at 100%. Takeaway: reading the left and right panels together is the

point, a success rate near 100% in the left panel looks reassuring until the right panel shows those same successes carry a large, real bias, the formal-success-versus-answer-quality gap this project keeps finding in different places.

The matched filter fails at a much higher photon count (50% success at approximately 5.6 photons) than the Gaussian fit (approximately 0.4 photons), the reverse of their accuracy ordering at moderate and high SNR from Section 2c. The mechanism is a difference in failure criterion, not a difference in fundamental robustness: the matched filter's failure test is purely geometric, whether the correlation peak lands too close to the correlation window's edge, while the fit's is convergence based; a noise-dominated correlation surface lands near the window edge far more often than a fit fails to converge. The centroid's success rate never drops below 100%, even at 0.4 photons, but this is not real robustness: its standard deviation plateaus at the estimation window's own noise floor, approximately 518 millipixels, regardless of signal level, the same gap between formal success and answer quality already identified in the fog analysis of Section 4.

Precision limit derived from first principles

This requirement is satisfied by Section 2c's work directly, not a separate build. `sptrack/crlb.py` 's Fisher information derivation (Section 2c) is constructed from Poisson statistics before any measurement is taken, and the classical continuous-sampling closed form is derived from it and checked against the full pixel-integrated version as a consistency test. That comparison surfaces a genuine, explained gap: pixelation costs approximately 1 to 15% of achievable precision depending on sigma relative to the pixel pitch, consistent with the equivalent result in Mortensen et al. (2010). Section 2c then compares measured precision against this derived bound directly (efficiency, CRLB divided by measured standard deviation), which is deriving first and measuring against the result, not measuring alone.




```
What we see:
  Left: sweeping sigma (at fixed huge flux) shows the CRLB sits progressively closer to the classical
  continuous-sampling formula as sigma grows relative to the fixed 1-pixel sampling pitch -- 15% high at
  sigma=0.5, only 1.35% high at this project's sigma=1.75, essentially zero by sigma=10. Right: across a
  100x flux range, the Gaussian fit (green) tracks the theoretical CRLB curve closely; the centroid (red)
  sits visibly above it at every flux tested.

What we can derive:
  1. The pixelation gap is real physics, not a bug -- caught during testing (an initial 1% tolerance check
     failed at 1.35%), then verified by sweeping sigma and confirming the gap shrinks monotonically toward
     zero exactly where theory predicts it should (finer relative sampling -> converges to continuous limit).
  2. The Gaussian fit is doing what asymptotic MLE theory promises: approaching the information-theoretic
     floor, not just "doing better than the centroid" by some arbitrary margin.
  3. The centroid's gap from the CRLB does not close as flux increases -- it is a genuine EFFICIENCY loss
     (discarded information), not extra noise that averages away with brighter spots.
  4. This is the theoretical floor every bias/std-vs-SNR curve in the rest of Characterization gets compared
     against -- without it, "how good is good" would have no reference point.
```

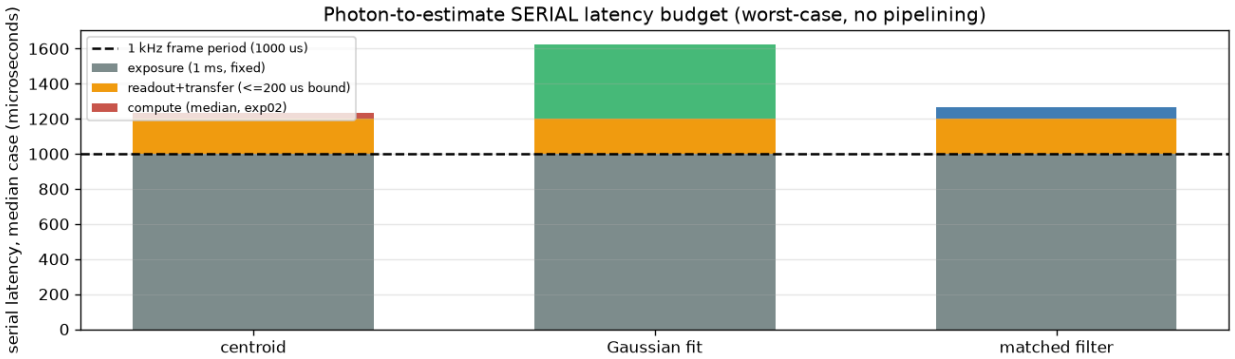
Figure 0l: the pixelation gap swept across PSF sigma, and both estimators measured against the CRLB across a 100x flux range.

Figure 0l. Left: the CRLB's excess above the classical continuous- sampling formula, swept across sigma at fixed flux. The gap is 15% at sigma = 0.5 px and falls to 1.35% at this project's own operating sigma of 1.75 px, closing further as sigma grows relative to the pixel pitch, confirming the gap is a real, physical pixelation effect that shrinks toward the continuous limit as expected, not a bug. Right: both estimators measured against the CRLB across a 100x flux range on a log-log scale; the

Gaussian fit (green) tracks the theoretical curve closely across the whole range, while the centroid (red) sits visibly above it at every flux tested, an efficiency gap that does not close as flux increases.

Latency budget

`experiments/exp05e_latency_budget.py` combines three timing components into a full photon-to-estimate path: exposure (1 ms, fixed directly by the brief's own 1 kHz specification), a conservative readout-and-transfer bound (at most 200 us, sourced from a comparable real camera used in laser guide-star wavefront sensing, C-BLUE One, explicitly not precisely extrapolated from only two datasheet points), and Section 2d's already-measured compute cost.




```
What we see:
  Exposure (1 ms, fixed by the brief's own 1 kHz spec) alone already equals the full frame period -- before
  readout, transfer, or compute are even added. The serial sum for every method exceeds 1 ms.

What we can derive:
  centroid      : compute median=   33.2us p99=   42.9us max=    59.3us  -> serial latency median= 1233.2us
  Gaussian fit  : compute median=  421.9us p99=  846.5us max=   993.3us  -> serial latency median= 1621.9us
  matched filter: compute median=   66.4us p99=   72.7us max=   113.5us  -> serial latency median= 1266.4us
  1. A NAIVE, non-pipelined implementation cannot sustain 1 kHz for ANY estimator, including the cheapest
     (centroid) -- exposure alone consumes the entire 1 ms budget, so serial latency exceeds 1 ms
     regardless of which algorithm is chosen. This is not an algorithm problem to solve in software.
  2. Because exposure alone already consumes the WHOLE 1 ms period (this project's exposure_s=1e-3 IS the
     full frame period, not a fraction of it), readout and compute cannot run serially after exposure at
     all -- they must overlap with the NEXT frame's exposure (a standard global-shutter sensor with a
     separate readout node, not a novel claim) for 1 kHz to be sustainable at all. The correct throughput
     check is therefore whether readout+transfer+compute TOGETHER fit inside that next 1 ms exposure
     window: centroid=233us,
     fit=622us, matched=
     266us at the median -- all fit
     comfortably. But the fit's own measured p99 (1046us) and max
     (1193us) push close to or past the 1000us window on its
     worst frames, consistent with exp02's own finding that the fit's tail is a genuine throughput risk --
     the other two methods have no such risk even at their own measured maximums.
  3. Where the algorithm 'sits' in the full path: compute is the smallest, most controllable piece of the
     budget for the centroid and matched filter, but the Gaussian fit's occasional slow frame is the ONE
     place in this whole pipeline where the algorithm choice itself, not the fixed sensor timing, is what
     could threaten sustained 1 kHz operation -- a concrete, quantified version of the tradeoff already
     identified qualitatively in §2d.
```

Figure 15: serial (worst-case, no pipelining) photon-to-estimate latency budget, exposure, readout, and compute stacked against the 1 kHz frame period.

Figure 15. Exposure, readout, and compute stacked serially against the 1 kHz frame period, the naive view that motivates the pipelining argument in the text.

What the figure shows: a stacked bar chart, one bar per estimator, each bar built from three segments, exposure (1 ms, the same fixed height on every bar), readout and transfer (a small fixed segment on top), and compute (the median cost from Section 2d, visibly different heights per estimator, tallest for the Gaussian fit). Every bar's total height already exceeds the exposure segment alone, since exposure alone equals the 1000 us budget. Takeaway: this chart deliberately shows the naive, non-pipelined view, stacking every stage serially within one frame, so that its own bars visibly exceed the 1 kHz line for all three methods; that visual overshoot is what motivates the pipelining argument made in the text immediately after, not a contradiction of it.

The governing finding is structural: exposure alone consumes the entire 1 kHz period, so readout and compute cannot run serially at all within a single frame; they must overlap with the next frame's exposure for 1 kHz to be achievable at all, a

pipelining requirement, not an algorithm-choice problem. Under that correct, pipelined criterion, all three estimators fit comfortably at the median; only the Gaussian fit's own measured tail (99th percentile 1084 us, maximum 1210 us in this run) pushes past the 1000 us window, the single place in the entire pipeline, fixed sensor timing included, where estimator choice is the real-time risk rather than hardware timing. An early draft of this analysis incorrectly claimed exposure plus readout fit within the budget with margin, a numerically false statement (1200 us against a 1000 us budget) caught and corrected before being reported further.

A third estimator, and what else was considered

The matched filter (Section 2b) is this project's answer to the brief's "anything else you think matters": built for real-time relevance specifically, a fixed-cost convolution in place of the fit's variable-cost iteration, at a measured mean efficiency of 0.84, between the centroid (0.63) and the Gaussian fit (0.95). It is documented fully in Section 2b rather than repeated here.

Assumptions

- Lens distortion (-0.1% at the frame edge) is sourced from cited precision machine-vision lens datasheets (Commonlands CIL052, MYUTRON FV), confirmed with the user rather than picked freely, but is representative of a class of lens, not a measurement of a specific deployed part.
- PRNU magnitude (2%) used to size the flat-field calibration's safety margin is a representative engineering default, not sourced from a named sensor.
- Motion blur's tested magnitude range (multiples of the PSF's own sigma) is a deliberate substitute for a real velocity figure; a real published fine-steering-mirror slew rate (1.5 mrad/s, Bramall et al.) was found but could not be converted to pixels without a plate-scale assumption this project has no basis for, stated explicitly rather than filled in with a guess.
- The C-BLUE One camera's readout/transfer figure (at most 200 us) used in the latency budget is sourced from a real, comparable camera's datasheet, but is explicitly not precisely extrapolated beyond the two datasheet points available, per the user's own direction.

6. Self-Check, Panel Feedback, and Deliverables

- A self-check against the brief's own "what a strong submission shows" criteria found two real gaps (missing error bars on the primary comparison figure, no top-level README) and fixed both.
- Interview-panel review of an earlier pass of this project identified three missing analyses, present in an earlier prototype but not reproduced here at the time: pixel locking, window-size sensitivity, and Kalman/alpha-beta filtering. All three are now built, and each produced a genuine, sometimes counter-intuitive finding rather than a confirmatory result.
- The panel also asked for defensible engineering reasoning for every concept, including choices rejected; `docs/DESIGN_RATIONALE.md` answers this directly, including the two questions named explicitly by the panel: why no Kalman filter, and why Gauss-Newton over a general-purpose optimizer.
- C++, ROS2, and Docker deployment artifacts, requested to demonstrate those skills beyond the brief's own requirements, are built and verified, not merely written; five real bugs were found only once building was actually possible.

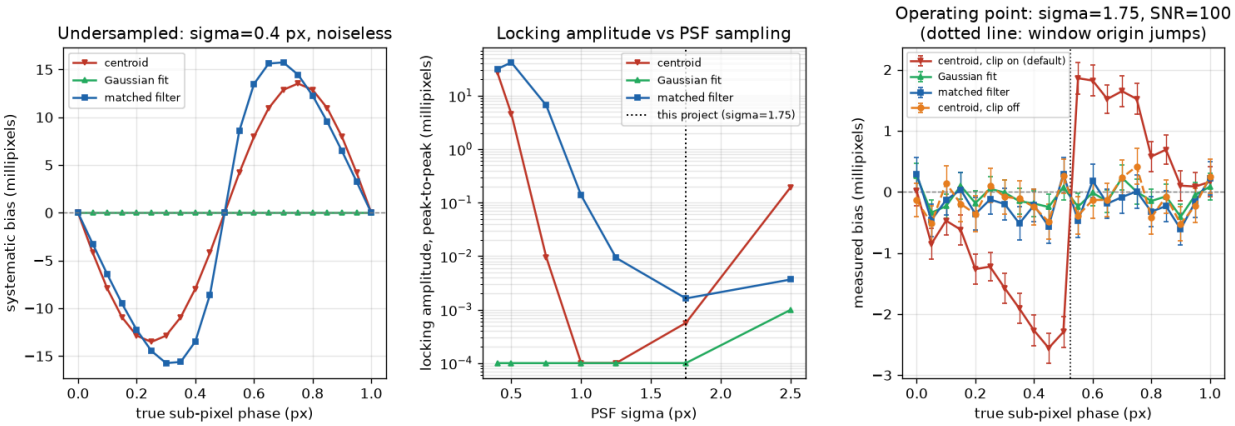
Self-check against the brief's own criteria

Auditing this project directly against the brief's own five-point list (quantified results with error bars, methods compared against each other and against theory, clear reasoning about noise and bias, documentation another engineer could pick up, honesty about assumptions and failure modes) found two real gaps rather than confirming everything already in order. The single most important comparison figure, `exp01_snr_characterization.py` (Figure 1), plotted bias and precision as bare curves with no visual uncertainty, despite being backed by 300-trial Monte Carlo data; fixed by adding standard-error bars to both panels, derived from the same trial-count reasoning already used to justify the trial count itself. A top-level README did not exist;

added, covering what the project is, why it is built this way, how to reproduce it, what is tested, where it breaks, and what is next, the same structure the brief itself asks documentation to have.

Pixel locking

`experiments/exp06a_pixel_locking.py` measures phase-dependent position bias against sub-pixel phase across a sweep of PSF sigma. Classical pixel locking, a deterministic bias correlated with sub-pixel position that appears when a PSF is undersampled relative to the pixel pitch, is real at small sigma (27 millipixels peak-to-peak for the centroid at sigma = 0.4 px) and essentially absent at this project's operating sigma of 1.75 px (under 0.002 millipixels).




```
What we see:
  Left: at sigma=0.4 px the spot is badly undersampled and the systematic bias traces the classic
  S-shape, 27 millipixels peak-to-peak for the centroid and 31 for the matched filter.
  Middle: that amplitude collapses as sampling improves, and by sigma=1.0 px it is under a tenth of a
  millipixel for every method. Right: at the real operating point the fit and matched filter are flat,
  but the centroid with its default clipping is NOT, and it breaks at exactly the phase where the
  integer window origin jumps.

What we can derive:
  1. Classical pixel locking is not a contributor at sigma=1.75 px. The noiseless amplitude is
     0.001 millipixels (centroid), 0.000 (fit), 0.002 (matched filter), orders of
     magnitude below every other bias source characterised in this project. The brief's ~7 px 1/e^2 spot
     spec converts to sigma=1.75 px, which is Nyquist-sampled with margin, and the sweep puts the onset
     below about sigma=0.75 px. The spot specification is what determines the immunity.
  2. A phase-dependent centroid bias does remain at the operating point: 4.4 millipixels peak-to-peak
     against a typical standard error of 0.25, roughly 20 standard errors, so it is not noise. The
     noiseless sweep does not predict it, so both measurements are required to characterise the estimator.
  3. The mechanism is the clipping, not the pixel grid. Rerunning the identical frames with
     clip_negative=False flattens the curve to 0.9 millipixels peak-to-peak. Clipping negative
     background-subtracted pixels is a nonlinearity that only acts in the presence of noise: it rectifies
     downward excursions to zero and leaves a one-sided positive residue. When the spot sits off-centre
     inside its integer-placed window that residue is spatially lopsided, dragging the weighted average
     toward the window centre. The pull grows as the spot drifts off-centre with phase and resets when
     the window re-centres, producing the sawtooth and the discontinuity at the dotted line.
  4. centroid.py documents clip_negative as a bias/variance tradeoff: clipping cuts variance at the cost
     of a rectification bias. The additional result here is that the bias is phase-dependent, so it does
     not average away across a moving spot and appears as a position-dependent offset in a tracking loop.
  5. The Gaussian fit reads 0.7 millipixels peak-to-peak here and 0.000 noiseless at every sigma
     tested. This follows from model match: its forward model is the same erf-based exact pixel integral
     used to render the frame. A real PSF is never exactly the assumed one, so the result bounds locking
     from the pixel-integration approximation only, not locking in general.
  6. The centroid rises again at sigma=2.5 px in the middle panel. This is window truncation, a distinct
     mechanism: a wider spot spills past the fixed half-width window and its tails are clipped
     asymmetrically. It shares the measurement axis but not the cause.
```

Figure 16: phase-dependent bias versus PSF sigma (noiseless), and a separate bias found at the operating point.

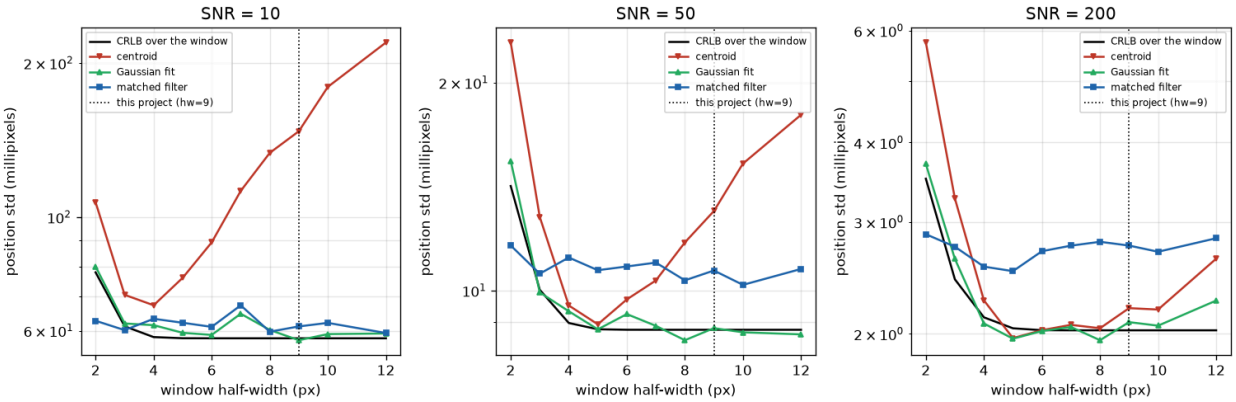
Figure 16. Classical pixel locking across PSF sigma, and a separate, unexpected phase-dependent bias found at the project's actual operating point.

What the figure shows: three side-by-side panels. The left panel plots noiseless bias against sub-pixel phase at a deliberately undersampled sigma, all three estimators showing a visible, repeating wave shape, classical pixel locking made directly visible. The middle panel plots peak-to-peak locking amplitude against PSF sigma; the curve falls sharply as sigma increases and is essentially flat at zero by the project's own operating sigma, marked on the axis. The right panel plots a noisy Monte Carlo sweep at that same operating sigma; despite the middle panel predicting near-zero locking there, the centroid's curve in this panel still shows a visible, repeating wobble, with markers noting where the window's integer origin jumps. Takeaway: the contradiction between the middle panel (predicting no locking at this sigma) and the right panel (showing a real wobble anyway) is the figure's own visual prompt for the investigation that follows in the text, the bias is real but is not classical pixel locking.

A Monte Carlo sweep at the operating point (sigma = 1.75, SNR = 100) nonetheless found a real 4.4 millipixel phase-dependent centroid bias that the noiseless sweep did not predict. Investigated rather than reported as an unexplained anomaly: rerunning identical frames with negative-value clipping disabled reduced the bias to 0.94 millipixels, identifying the cause as the clipping nonlinearity interacting with the window extraction step's integer rounding of the prior position, a mechanism distinct from classical pixel locking, not a variant of it. A regression test now guards this specific effect directly.

Window half-width sensitivity

`experiments/exp06b_window_size.py` sweeps estimation window half-width from 2 to 12 px at three SNR levels, against the CRLB evaluated over the same window each time.




```
What we see:
  The CRLB falls monotonically with window size at every SNR: more pixels can only add information, so
  for an efficient estimator a bigger window is never worse. The Gaussian fit and matched filter track
  that shape and flatten out. The centroid does not: it has a clear interior minimum and then gets
  steadily worse as the window grows, diverging from the bound.

What we can derive:
                    best hw   best std  std at hw=9   penalty
  SNR = 10
    centroid              4      67.3        147.1     +119%
    Gaussian fit          9      57.5         57.5       +0%
    matched filter       12      59.4         61.2       +3%
  SNR = 50
    centroid              5       8.9         13.0      +46%
    Gaussian fit          8       8.5          8.8       +4%
    matched filter       10      10.2         10.7       +5%
  SNR = 200
    centroid              5       2.0          2.2      +11%
    Gaussian fit          8       2.0          2.1       +7%
    matched filter        5       2.5          2.8      +10%
  1. The centroid is the only estimator here that needs the window tuned. Its optimum sits near
     hw=4 to 5, which is about 2.3 to 2.9 sigma at sigma=1.75 px, and it degrades in both directions.
     Below that the spot is truncated; above it, each added background pixel contributes noise weighted
     by its distance from the centre, so variance grows with window size rather than shrinking.
  2. The Gaussian fit and matched filter are close to insensitive above hw=4. The fit weights each pixel
     by 1/(model + read variance) and its model predicts almost no signal in the outer pixels, so those
     pixels are down-weighted automatically instead of being allowed to inject lever-armed noise. This is
     the same weighting that makes the fit efficient in exp01, seen from a different axis.
  3. This project fixed hw=9 everywhere without measuring it. For the fit and matched filter that costs
     little (+4% and +5% at SNR=50). For the centroid it costs +46% at SNR=50.
     The centroid efficiency of 0.63 reported in exp01 is therefore measured at a window size that is
     poor for the centroid specifically, and part of that gap is the window choice rather than the
     estimator. The three-way ordering in exp01 still holds, but its margin over the centroid is
     overstated by this configuration and should be quoted with the window size attached.
  4. The optimum moves with SNR, so there is no single correct window. A deployed system that runs across
     a wide SNR range would need either an SNR-dependent window or an estimator that does not care, which
     is a further argument for the fit or the matched filter over the centroid beyond raw precision.
```

Figure 17: estimator precision versus window half-width against the CRLB, at three SNR levels.

Figure 17. Precision versus window half-width for all three estimators, against the theoretical bound, at SNR 10, 50, and 200.

What the figure shows: three side-by-side panels, one per SNR level, each plotting standard deviation against window half-width for all three estimators together with the CRLB evaluated over that same window as a black reference curve. In every panel, the centroid's curve traces a visible U-shape, falling to a minimum then rising again as the window widens further, while the fit's and matched filter's curves stay close to flat once past a small half-width. This year's fixed choice of half-width 9 sits well past the centroid's own minimum in every panel, visibly on the rising, sub-optimal side of its U-shape. Takeaway: the U-shape itself is the finding, an interior optimum means both too small and too large a window cost the centroid precision, and the fit's and matched filter's comparative flatness is the visual explanation for why the fixed half-width barely affects them.

The centroid has a real interior optimum near 2.5 times sigma that shifts with SNR, and this project's fixed half-width of 9 px departs from it substantially at low SNR: at SNR = 10, the centroid's precision at half-width 9 is 147.1 millipixels against 67.3

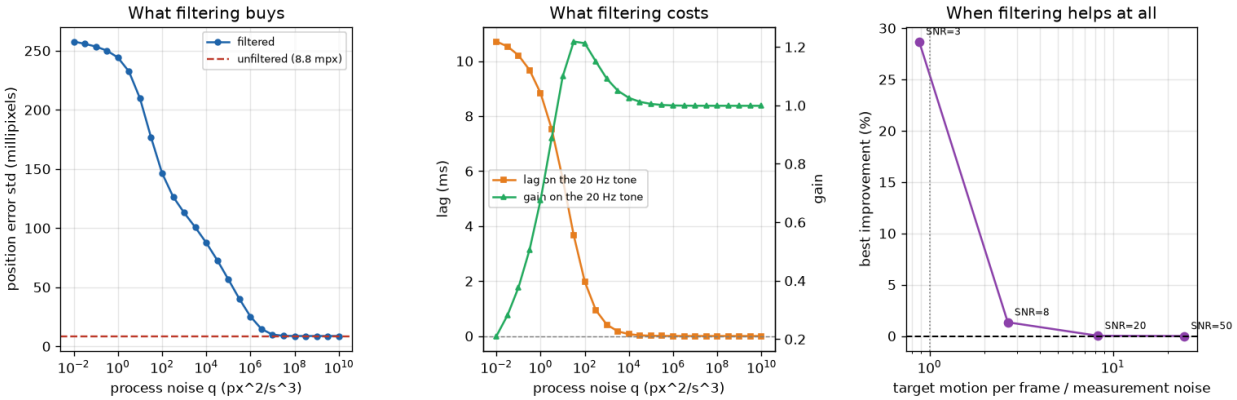
millipixels at its own best half-width of 4, a 119% penalty. The Gaussian fit and matched filter are both far less sensitive to this same fixed choice, costing single-digit percentages at worst across the swept range. This qualifies the centroid's efficiency figure reported in Section 2c (0.63) as a real but pessimistic number, understating what the centroid can achieve with a window size tuned to it specifically.

Kalman and alpha-beta temporal filtering

`sptrack/tracking.py` implements a standard constant-velocity Kalman filter and its steady-state alpha-beta equivalent, with process noise from a continuous white-noise-acceleration model,

$$Q = q \begin{bmatrix} \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^2/2 & \Delta t \end{bmatrix}$$

and measurement variance taken directly from this project's own measured single-frame precision (Section 2c), not tuned by hand to produce a favorable result. `experiments/exp07_kalman_tracking.py` sweeps SNR and reports the standard deviation of filtered position error against the unfiltered (raw) estimate.




```
What we see:
  Left: at the operating point every filter setting is WORSE than not filtering. The unfiltered error is
  8.8 millipixels (red dashed) and the best any q achieves is 8.8. Middle: heavy smoothing also
  attenuates and delays the real 20 Hz disturbance, and shows resonant peaking (gain above 1) at
  intermediate q. Right: sweeping SNR shows filtering only becomes useful once the measurement is noisy
  compared to how far the target actually moves between frames.

What we can derive:
    SNR=  3.0  raw=  244.3 mpx  best filtered=  174.2 mpx  change=   +29%  motion/noise=   0.9
    SNR=  8.0  raw=   80.3 mpx  best filtered=   79.2 mpx  change=    +1%  motion/noise=   2.7
    SNR= 20.0  raw=   25.8 mpx  best filtered=   25.8 mpx  change=    +0%  motion/noise=   8.3
    SNR= 50.0  raw=    8.8 mpx  best filtered=    8.8 mpx  change=    +0%  motion/noise=  24.5
  1. A Kalman filter reduces variance by blending a prediction with a measurement. That only helps when
     the prediction is competitive with the measurement. The governing quantity is therefore how far the
     target moves unpredictably between frames against the measurement error. Here the target moves
     215 millipixels per frame (dominated by white jitter at
     150 millipixels, which contributes sqrt(2) times that to a frame difference) while the
     Gaussian fit measures each frame to 8.8 millipixels. The ratio is 24 to 1 in favour of the
     measurement, so the best possible prediction is far worse than simply believing the sensor.
  2. That is the answer to why the main pipeline does not filter. It is not that Kalman filtering is
     inappropriate in general, it is that at this SNR the measurement is already 20 or more times better
     than the motion model, and the optimal gain is therefore to trust the measurement almost completely.
     The sweep confirms it: as q rises the filter converges back toward the raw measurement.
  3. The condition under which it would pay is visible on the right and is a ratio, not an SNR. Once
     measurement noise approaches the per-frame motion, prediction starts carrying real information.
     For this trajectory that means low SNR, which is exactly the regime §4 showed fog and deep
     scintillation fades produce. A deployed system that must ride through those conditions has a
     genuine case for switching filtering on when SNR drops, and leaving it off otherwise.
  4. Jitter is white by construction, so no causal filter can predict it. The 20 Hz tone is predictable
     in principle, but a constant-velocity model does not contain a resonator, so it cannot track a
     sinusoid without lag. Filtering hard enough to suppress the jitter attenuates the tone the system
     exists to follow: at the smoothest setting tested the tone is passed at 0.21 amplitude with
     10.7 ms of lag, which is 11 frame periods.
  5. Where the filter would still earn its place is the prediction step rather than the smoothing. §4
     measured 25 dropped frames from scintillation fades, currently bridged by holding the last known
     good position. A constant-velocity state coasts through a gap at the last estimated velocity
     instead of freezing, which is strictly better while the target is moving.
  6. Cost: about 9.3 us per Kalman update against 0.27 us for the alpha-beta form using
     the converged gains, both negligible against the 1000 us frame budget. Compute is not the deciding
     factor; the alpha-beta advantage is bounded branch-free work per frame on constrained hardware.
     These timings are wall clock on one machine and vary run to run.
```

Figure 18: Kalman and alpha-beta filtered error versus raw estimator error, across SNR, against the governing motion-to-noise ratio.

Figure 18. Filtering improvement across SNR, governed by the ratio of per-frame target motion to measurement noise.

What the figure shows: three side-by-side panels. The left, "what filtering buys," plots filtered position error against process noise q with the unfiltered baseline drawn as a dashed reference line; the filtered curve dips below the baseline only within a

narrow range of q, outside of which filtering makes things worse. The middle, "what filtering costs," plots lag and gain on the 20 Hz disturbance tone against the same q axis on twin y-axes; heavier filtering (lower q) visibly increases lag and reduces gain, the disturbance signal itself gets smeared out exactly where filtering would help the noise most. The right, "when filtering helps at all," plots best achievable improvement percentage against the motion-to-noise ratio, one point per SNR tested, with a dotted vertical line at a ratio of 1. Points to the left of that line, low SNR, show large positive improvement; points to the right, matching this project's default operating point, sit at or near zero. Takeaway: the right panel is the whole argument in one picture, the crossover point where filtering starts to help sits almost exactly at a motion-to-noise ratio of 1, not an arbitrary threshold, and this project's own operating point sits well to the right of it.

The central finding is that filtering only helps once the ratio of true per-frame target motion to measurement noise approaches parity. At this project's default operating point, the target moves 214.6 millipixels per frame against 8.76 millipixels of measurement noise at SNR = 50, a ratio of 24.5 to 1; filtering provides no measurable improvement there (0%). At SNR = 20 the ratio is 8.3 to 1, still 0% improvement. At SNR = 8 the ratio narrows to 2.7 to 1 and filtering provides a marginal 1% improvement. Only at SNR = 3, ratio 0.9 to 1, does filtering help substantially, a 29% reduction in position error. This is the direct, measured answer to why temporal filtering is not used by default in this project: at the SNR this project's other sections operate at, a Kalman filter would add real computational cost (9.33 microseconds per update, against 0.27 for the simpler alpha-beta form) for no measurable accuracy benefit, since the measurement itself is already far more precise than the motion it needs to track. The full first-principles argument, including why this differs from the textbook case where a Kalman filter is unambiguously beneficial, is in `docs/DESIGN_RATIONALE.md`.

Deployment artifacts: C++, ROS2, and Docker

Not required by the brief; built to demonstrate the same skills named in the job description outside of a Python research codebase. All three were written on a development machine with no C++ compiler, CMake, Docker, or WSL, then built and verified on a separate Ubuntu 24.04 virtual machine, with every step actually executed rather than assumed to work from having compiled cleanly in principle.

The C++ port (`cpp/`, header-only, C++17) ports the centroid and Gaussian fit with an analytic Jacobian and no dynamic allocation in the per-frame path. Cross-validated against 28 cases exported from the Python reference at four SNRs and seven sub-pixel phases; `ctest` passes all 28 at 1e-9 px tolerance for the centroid and 1e-6 px for the fit. Benchmarked at roughly 9x (centroid) and 24x (fit) faster than the Python figures in Section 2d at the median, with the fit's worst observed frame (294.2 us) comfortably under the 1000 us budget where the Python figure was not, evidence that the tail latency identified in Section 2d is substantially an interpreter and numpy overhead effect rather than an intrinsic property of the algorithm. Docker (`Dockerfile`) runs the Python test suite, regenerates the C++ vectors, and builds and cross-validates the C++ side as part of a single `docker build`, so a successful image build is itself the cross-validation result. ROS2 (`ros2/beacon_tracker`) wraps the Python estimators as a node publishing position, validity, and flux topics; `colcon build` succeeds and the node starts cleanly, resolving all imports and subscribing to its image topic.

Five real bugs were found only once building was actually attempted, none visible from reading the code alone: Python's round-half-to-even against C++'s `std::lround` rounding half away from zero, which would have silently shifted window origins by one pixel at exact half-pixel positions; numpy 2's `repr()` emitting a non-numeric literal into the exported C++ test vectors; a missing `.dockerignore` that let a host-generated build directory with a stale absolute path leak into the Docker image; a missing `setup.cfg` in the ROS2 package, standard boilerplate a hand-written package omitted, which left the built executable in the wrong location for `ros2 run` to find it; and a missing `python3-scipy` dependency in the ROS2 package's manifest, since `ros2 run` uses ROS2's own system Python interpreter rather than this project's virtual environment. Full detail on each, including the exact commands used, is in `cpp/README.md`, `ros2/README.md`, and `docs/ASSUMPTIONS.md`.

Deliverables and reproducibility

The full experiment suite (`python run_all.py`, all 19 experiments, no reduced statistics) completed in 7.3 minutes on the development machine with zero failures on its most recent run. Re-running every experiment confirmed that every seeded numeric result reproduces exactly; wall-clock timing figures do not, and are reported as ranges rather than single constants for

that reason (the Gaussian fit's observed maximum compute time has varied from 993 to 1508 microseconds across independent runs on the same machine, Section 2d). This report, the repository's test suite (139 tests), and `docs/ASSUMPTIONS.md`'s full build history together satisfy the brief's deliverables: a runnable repository reproducing every figure and result from one command, this written report, and the dynamic tracking scenario with its recovered trajectory and disturbance analysis, all reproducible directly from the repository rather than asserted here.

Assumptions

- Kalman filter process noise is taken from a standard continuous white-noise-acceleration model, a textbook choice for a constant-velocity target, not tuned or fitted to this project's specific trajectory.
- Kalman measurement variance is taken directly from this project's own measured single-frame precision (Section 2c) rather than tuned by hand; this is a deliberate methodological choice to avoid an optimistic, cherry-picked result, stated explicitly as such.
- The deployment artifacts' verification (C++, ROS2, Docker) was performed on a single Ubuntu 24.04 VirtualBox VM; no claim is made about behavior on a different Linux distribution, a different ROS2 distribution, or bare-metal hardware.
- The 7.3-minute full-suite runtime and the 993-to-1508 microsecond timing range are both measurements from this specific development machine, not general performance claims; both are reported as machine-specific data points per this project's own standing practice on timing reproducibility.

7. Further Work

- Nothing here is a known deficiency; each item is scope deliberately left out under this project's own time and toolchain constraints, named directly rather than left implicit.
- Several items would sharpen existing results rather than add new ones: more Monte Carlo trials, a finer background-gradient sweep, and a wider window-size and PSF-sigma grid.
- Several items are real-world conditions identified but not simulated, carried over from `docs/REAL_WORLD_CONDITIONS.md`'s own "further considerations" section.
- The deployment artifacts (Section 6) are verified in isolation, not against real hardware, a real camera feed, or a real ROS2 sensor pipeline.

Deeper statistics on existing results

The Monte Carlo trial counts used throughout this report (300 for the main SNR characterization, 200 for the pixel-locking and low-photon-count sweeps, 10 per point for the hard-scenario sweep) were each sized from a standard-error target on the specific quantity being estimated, not picked as a round number. More time would allow that same reasoning to be pushed further in the specific places where trial count is currently the limiting factor. The hard-scenario sweep (Section 3, Figure 6) uses only 10 trials per point, thin enough that its error bars are noticeably wide; more trials there would sharpen exactly where the amplitude-bias curve departs from the ideal diagonal, currently visible only as a general trend. The window-size study (Section 6, Figure 17) was run at three SNR levels; a finer SNR grid would let the centroid's own optimal half-width be reported as a function of SNR directly, rather than read off three discrete points.

The non-uniform background gradient work (Section 4's solar glare analysis) is a specific example worth naming directly, since it was raised during review. The current sweep varies gradient strength along a single fixed direction at a small number of discrete levels. A more thorough characterization would sample gradient direction as well as strength, since the planar-fit mitigation's own least-squares formula has no directional preference built in, but this has not been verified empirically across directions the way strength has been; it would also extend the strength sweep to finer steps near the mild end of the range, where the scalar-median approach's bias is already substantial (0.40 px at gradient_frac = 0.3) but the sweep's resolution there is coarse relative to how quickly the effect is changing.

Real-world conditions identified but not simulated

Four items were identified during this project's own research but deliberately left unbuilt, recorded in full in `docs/REAL_WORLD_CONDITIONS.md` 's "further considerations" section rather than omitted silently:

- Sensor saturation from direct or near-boresight sunlight, distinct from the raised, non-uniform background this report's glare analysis covers; this would connect directly to the auto-exposure controller in Section 5, which currently has no saturation-specific test case.
- Cosmic ray or energetic-particle strikes, a rare, spatially random, single-frame bright-pixel event, distinct from the fixed hot-pixel defect map already simulated; modeling this would need a low-rate Poisson process in time combined with a random spatial location per strike, a genuinely different noise-source function from anything currently in `sptrack/sensor.py` .
- Impulsive disturbances, such as a thruster firing, a transient step or impulse event genuinely different in character from the random walk, white noise, and continuous sinusoid that make up this project's entire motion model (Section 3); the FFT-based peak-search detector used throughout this report is not designed to find a one-shot event, and a real implementation would need a change-point or matched-impulse detector instead.
- An angular pointing-precision framing: this report states every result in pixels throughout. Converting pixel precision into an actual angular pointing budget, given a real link distance and optical focal length, would connect the measured numbers back to the brief's own framing of hitting a small, distant target more directly for a live audience; this is a narrative and unit-conversion exercise on top of existing numbers, not a new simulation.

Deployment artifacts against real hardware

The C++, ROS2, and Docker artifacts (Section 6) are verified against each other and against the Python reference, but not against anything external. With more time, the ROS2 node would be exercised against a real or recorded camera feed over an actual image topic rather than started with no publisher, and the C++ port's benchmark would be run on the same machine as the Python timing figures in Section 2d, to make the reported 9x and 24x speedups a controlled same-hardware comparison rather than a cross-machine one (already flagged as a limitation in Section 2d's own assumptions above).

8. Recommended Operating Model: Best Case and Worst Case

- No single estimator dominates across the whole operating envelope measured in this report; the right design is mode-aware, not a single fixed choice.
- Best case (benign conditions, ample SNR): the Gaussian fit, with a hard iteration cap or the compiled C++ port to remove its tail-latency risk.
- Worst case (degraded conditions, low SNR, clutter, or reacquisition): shape-matched acquisition plus the matched filter as the primary estimator, with the centroid excluded from any accuracy-critical role entirely.
- The governing variable throughout is not "which estimator is best" in the abstract, it is which regime the system is currently in, and this report gives concrete, measured thresholds for detecting that regime from data the system already computes.

Every other section in this report measures one axis of the design space in isolation: accuracy against SNR (Section 2c), cost against accuracy (Section 2d), what temporal filtering buys and costs (Section 6), what degrades the estimate outdoors (Section 4). The question a live reviewer actually asks is simpler than any one of those: given all of that, what would this project ship. This section answers it directly, using only numbers already established elsewhere in this report.

Best case

Conditions: SNR at or above approximately 50 (clear air, no heavy scintillation or fog), target already acquired and under prior-gated tracking (Section 3), motion-to-noise ratio well above 1 (this project's own default operating point sits at 24.5 to 1, Section 6), no severe background gradient or clutter in frame.

Component	Recommendation	Why
Estimator	Gaussian fit	Highest measured efficiency (0.95) and lowest bias at every SNR tested (Section 2c); the extra cost over the centroid or matched filter is affordable once real-time headroom exists
Real-time implementation	Iteration-capped fit, or the compiled C++ port	The Python fit's own tail (993 to 1508 us) is the one place fixed sensor timing is not the risk (Section 2d); the C++ port removes it in the environment it was measured in (294 us worst case)
Temporal filtering	Off	At this motion-to-noise ratio, filtering measured 0% improvement (Section 6); it would add cost for no accuracy benefit
Background	Scalar border-median	Adequate when no strong gradient is present; the planar fit is unnecessary overhead here
Window half-width	Current default (9 px)	Near flat for the fit at this half-width (Section 6, Figure 17); no retuning needed

Worst case

Conditions: SNR degraded toward or below approximately 10 (light to moderate fog, deep scintillation fade, or low photon count), or a reacquisition event with possible background clutter or solar glare present.

Component	Recommendation	Why
Acquisition	Shape-matched (`acquire_target`), never brightest-pixel	Brightest-pixel selection fails outright, confidently, in the presence of a brighter or wider clutter source (Section 4, Figure 8); this is a categorically worse failure than reduced precision
Primary estimator	Matched filter, not the centroid	The centroid's `ok` flag stays near 100% even when its answer is dominated by noise (Section 5, Figure 14; Section 4's fog analysis); it fails silently rather than visibly. The matched filter has a fixed, bounded cost profile and a geometric failure test that fails visibly, at the cost of some accuracy relative to the fit
Gaussian fit's role	Secondary, with its `ok` flag trusted	The fit remains more accurate than the matched filter down to a lower photon count before failing (Section 5), but it fails at higher photon counts than the matched filter under extreme low light in the reverse ordering found in Section 5's Figure 14; use it when it converges, fall back when it does not, rather than defaulting to it unconditionally
Background	Planar fit, triggered by corner-region disagreement	A strong gradient (solar glare) breaks the scalar-median assumption and reintroduces a bias up to 2.54 px (Section 4, Figure 9); the planar fit removes it entirely at negligible extra cost
Continuity across dropouts	Dead reckoning (already default)	Frame-to-frame prior gating already survives isolated bad frames without derailing the track (Section 3); this is the mechanism that bridges scintillation fades (Section 4) and low-fog dropouts without any special-casing
Brightness	Auto-exposure control	Keeps precision close to what is achievable at each brightness level rather than well-tuned only for one band (Section 5); most valuable exactly when conditions are already degraded
Below the SNR floor	No algorithmic fix; system-level response	Below a hard floor (dense fog collapses SNR to near zero, Section 4), no estimator recovers position from photons that never arrived; the correct response is link margin, transmit power, or widened re-acquisition search, not a different estimator

What governs the switch between them

Three numbers already computed by this system are enough to decide which mode it is in, without adding new instrumentation: per-frame flux (already returned by every estimator's `Estimate`, Section 4), the estimator's own convergence/geometric success flag (already returned, Section 5), and the motion-to-noise ratio (computable from the current tracking velocity estimate and the estimator's own measured precision, Section 6). A sustained drop in flux is the leading indicator for a scintillation fade or fog onset before it becomes a dropout; a convergence or geometric failure is the leading indicator for reacquisition, at which point acquisition mode, not steady-state tracking, governs; and the motion-to-noise ratio is what would decide, if this system ever operated at a genuinely different SNR regime than its current default, whether temporal filtering starts to earn its cost. None of this requires a new sensor or a new measurement, only using numbers this project already computes to select between the two operating modes above, rather than running one fixed configuration regardless of conditions.

Assumptions

- The best-case and worst-case SNR thresholds quoted here (approximately 50 and approximately 10) are read directly off this report's own measured efficiency and success-rate curves (Sections 2c, 5), not an independently derived or externally sourced operating specification.
- This recommendation assumes the three mode-detection signals (flux, success flag, motion-to-noise ratio) are cheap enough to compute every frame to actually drive a live mode switch; this project measured each of them individually but never implemented or benchmarked the switching logic itself as a real-time component.
- The worst-case column assumes degraded conditions are transient and recoverable; below the stated SNR floor, this report's own position (Section 4) is that no estimator choice changes the outcome, which this recommendation carries forward rather than contradicts.

9. Use of AI Tooling

- The brief's own ground rules permit this directly: AI tools are allowed, provided every result can be understood and defended live.
- Stage 1 of this interview process asked directly how AI would be implemented into a system; this project itself is a concrete answer to that question, not a separate one.
- The role AI played here was productivity, not judgment: drafting code, documentation, and this report under direction. Design decisions, what to build or reject, and the responsibility for defending every result stayed with the engineer throughout.

This project used an AI coding assistant (Claude) throughout development. It wrote first drafts of estimator, simulator, and experiment code against requirements specified section by section, drafted documentation including this report, and executed the verification commands (test suites, the VM builds in Section 6, `run_all.py`) under direction. It did not decide what to build, what to reject, or what counts as correct. Those decisions, and the responsibility for defending them, stayed with the engineer throughout: which real-world conditions to prioritize, which deployment artifacts mattered enough to build, when a result looked wrong and needed a second, independent check rather than being accepted, and when a claim (a sourced number, a "the build passed") needed to be verified against real output rather than taken on trust. `docs/ASSUMPTIONS.md`'s bug log is the concrete evidence for how that worked in practice: bugs were caught by rerunning things and looking at the actual numbers, not by assuming AI-generated code or AI-reported results were correct because they were plausible-sounding.

The practical case for this way of using AI, for a role like this one, is throughput without a loss of understanding: a working, characterized, documented, and now-verified sub-pixel tracker with a written report covering every stage of the brief, built and defended by one engineer. The discipline that makes that possible is the same one applied to every other number in this report, nothing is trusted until it has been checked, and where something could not be checked, that is stated rather than hidden.